



数据挖掘与机器学习 作业汇总

信管2301

冯世欣

202321054001



数据挖掘与机器学习 第一次作业

课程导言

读书笔记

一、数据科学研究的四个核心步骤

准备数据 → 2. 选择算法 → 3. 参数调优 → 4. 评价模型

二、数据准备核心要点

维度	要点
数据格式	表格：行=数据点，列=变量
变量类型	①二值 ②分类 ③整型 ④连续
特征工程	重新编码、降维（合并相关变量）
缺失值处理	①众数/中位数近似 ②算法计算 ③最后考虑移除

读书笔记

三、选择算法

三大任务类型及其对应算法

任务类型	说明	对应算法
无监督学习	找出数据中隐藏的模式	k均值聚类、主成分分析、关联规则、社会网络分析
监督学习	使用数据中的模式做预测	回归分析、k最近邻、支持向量机、决策树、随机森林、神经网络
强化学习	根据反馈不断改进	多臂老虎机

读书笔记

四、参数调优

问题	表现	后果
过拟合	对数据中的随机波动过度敏感，误认为是持久模式	对当前数据预测准确，但对未知数据预测差（泛化能力弱）
欠拟合	过于愚钝，忽视数据中的基本模式	对当前和未知数据的预测准确度都差
特征工程	在识别主要趋势和忽视微小变化间找到平衡	模型非常适合做预测

防止过拟合的方法

- 正则化：引入惩罚参数，人为增大预测误差，对模型复杂度增加进行惩罚
- 使模型保持简单有助于提高泛化能力

读书笔记

五、评价模型

5.1 分类指标

① 预测准确率

- 最简单的定义：正确预测所占的比例
- 缺点：无法得知预测误差是如何产生的

② 混淆矩阵

	预测会买	预测不会买
实际买	正例	假负例
实际未买	假正例	负例

5.2 回归任务评价

均方根误差：误差取平方 → 放大误差，对异常值敏感

5.3 验证

① 数据集划分

训练集：用来生成和调整预测模型

测试集：充当新数据，评估模型预测准确度 需随机分配数据点以保证验证有效

② 交叉验证（适用于小数据集）

目的：最大限度利用可用数据

方法：

- 数据集划分成若干组，反复测试
- 除一组外，其他各组用于训练
- 留下的一组用于测试
- 重复直到每组都测试过一次
- 最终预测准确度 = 所有结果的平均值

概念汇总--大数据、数据挖掘

大数据

又称海量数据，指的是以不同形式存在于数据库、网络等媒介上蕴含丰富信息的规模巨大的数据

大数据的基本特征（4V）：

Volume：数据体量巨大。

Variety：数据类型繁多。

Value：价值密度低。

Velocity：处理速度快。

体量
大
，
结
构
杂
，
场
景
多

数据挖掘（Data Mining，DM）

也叫**数据开采**、**数据采掘**等，

①是从大量的、不完全的、有噪声的、模糊的、随机的实际应用数据中，②提取③隐含在其中的、人们事先不知道的，但又是④潜在有用的信息和知识的过程

概念汇总—数据降维 有监督、无监督

数据降维

大数据的一个特点是量可能很大，这样就可能超过计算机的处理能力，所以在对数据进行处理后，通常要考虑将数据进行降维，从而**缩减数据量**。

大数据降维的要点是根据数据挖掘的目标、数据量、计算机的处理能力、对时间的要求等多方面的因素，对数据进行分级降维，首先是通过抽样的方式对数据进行降维，第二层次是抽取有用的变量，第三层次根据经典的降维方法，如PCA等，进行数据的变形降维。这是一种分级形式的逐层降维方式。

另外一种降维方式是分散的降维方式，就是将大数据需要映射为小的单元进行计算，再对所有的结果进行整合，就是所谓的 map-reduce 算法框架。

无监督学习

任务目标：指出数据中隐藏的模式。当希望找出数据集中隐藏的模式时，可以使用均值聚类，主成分分析，关联规则，社会网络分析等无监督学习算法。之所以称之为无监督学习算法，是因为我们不知道要找的模式是什么，而是要依靠算法从数据集中发现模式。

监督学习

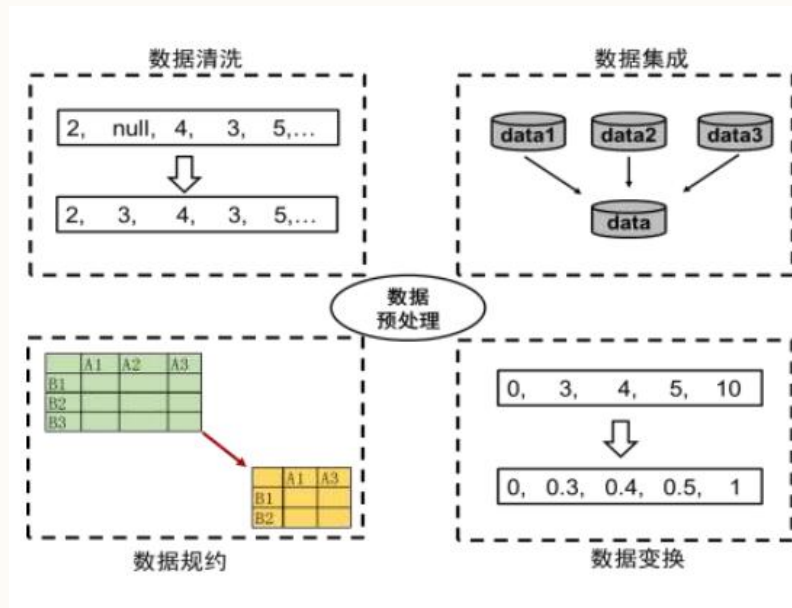
任务目标：使用数据中的模式做预测。当需要做预测时，就会用到回归分析，最近邻，支持向量机，决策树，随机森林，神经网络等监督学习算法。之所以称之为监督学习算法，是因为它们的预测都基于已有的模式。

概念汇总—— 结构化数据、半结构化数据、非结构化数据

- **结构化数据**，简单来说就是数据库。结合到典型场景中更容易理解，比如企业 ERP、财务系统、医疗 HIS 数据库、教育一卡通、政府行政审批和其他核心数据库等。这些应用需要哪些存储方案呢？其基本包括高速存储应用需求、数据备份需求、数据共享需求以及数据容灾需求。
- **非结构化数据**，包括视频、音频、图片、图像、文档、文本等形式。具体到典型实例中，如医疗影像系统、教育视频点播、视频监控、国土GIS、设计院、文件服务器（PDM/FTP）、媒体资源管理等具体应用，这些行业对于存储需求包括数据存储、数据备份以及数据共享等。
- **半结构化数据**，包括邮件、HTML、报表、资源库等，典型场景如邮件系统、WEB 集群、教学资源库、数据挖掘系统、档案系统等，这些应用对应于数据存储、数据备份、数据共享以及数据归档等基本存储需求

概念汇总—数据预处理、数据清洗

- **数据预处理**的主要任务可以概括成四个内容，即**数据清洗**、**数据集成**、**数据归约**和**数据变换**，如右图所示。
- **数据清洗**是通过填写缺失的值、光滑噪声数据、识别或删除离群点，并解决不一致性等方式来“清理”数据。如果用户认为数据是脏的，则他们可能不会相信这些数据上的挖掘纯果。此外，脏数据可能使挖掘过程陷入混乱，导致不可靠的输出。



概念汇总--数据标准化、数据归一化、衍生变量

数据的标准化 (Normalization) 是将数据按比例缩放, 使之落入一个小的特定区间。在某些比较和评价的指标处理中经常会用到, 去除数据的单位限制, 将其转化为无量纲的纯数值, 便于不同单位或量级的指标能够进行比较和加权。其中最典型的就是0-1 标准化和Z-score 标准化:

(1) 0-1标准化 (0-1 Normalization)

也叫离差标准化, 是对原始数据的线性变换, 使结果落到 $[0, 1]$ 区间, 转换函数如下:

$$x^* = \frac{x - \min}{x - \max}$$

其中, $\max$ 为样本数据的最大值; $\min$ 为样本数据的最小值。这种方法有一个缺陷就是 当有新数据加入时, 可能导致 $\max$ 和 $\min$ 的变化, 需要重新定义。

(2) Z-score 标准化 (Zero-mean normalization)

也叫标准差标准化, 经过处理的数据符合标准正态分布, 即均值为0, 标准差为1, 也是最为常用的标准化方法, 其转化函数为:

$$x^* = \frac{x - u}{x - \sigma}$$

其中, u 为所有样本数据的均值; σ 为所有样本数据的标准差。

衍生变量, 顾名思义是由其他既有变量通过不同形式的组合而衍生出的变量。例如, 已经知道一个长方形物体的质量、长度、体积, 现在就可以通过对现有三个变量的组合得到一些有用的衍生变量, 如密度=质量/体积, 线密度=质量/长度。



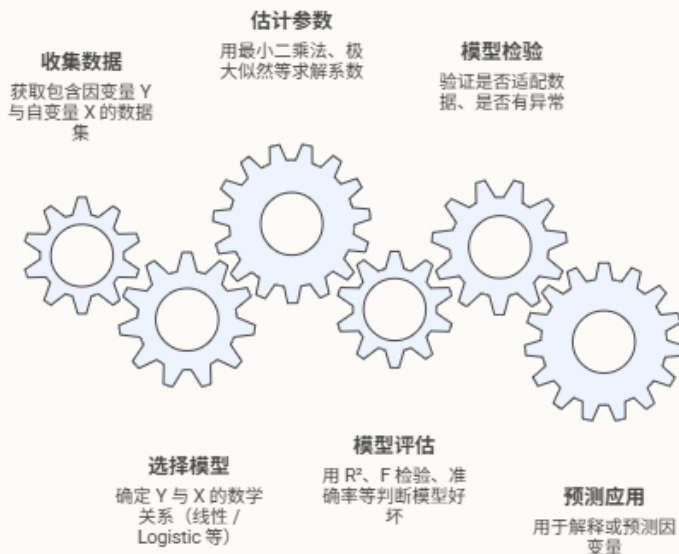
数据挖掘与机器学习 第二次作业

回归

1. 数据回归方法的基本方法、步骤

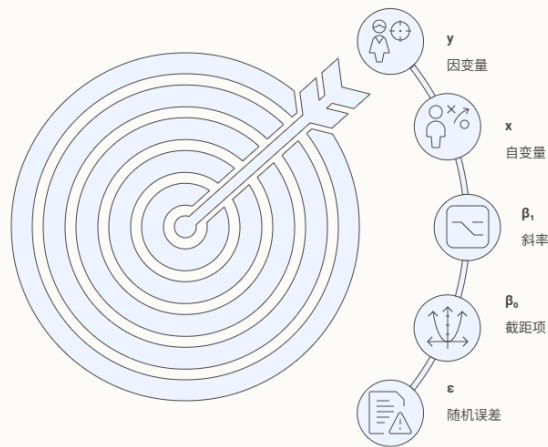
- **收集数据**：获取包含因变量 Y 与自变量 X 的数据集
- **选择模型**：确定 Y 与 X 的数学关系（线性 / Logistic 等）
- **估计参数**：用最小二乘法、极大似然等求解系数
- **模型评估**：用 R^2 、F 检验、准确率等判断模型好坏
- **模型检验**：验证是否适配数据、是否有异常
- **预测应用**：用于解释或预测因变量

回归分析过程



2. 一元线性回归基本数学模型

一元线性回归模型



$$y = \beta_0 + \beta_1 x + \epsilon$$

y : 因变量 (预测目标)

x : 自变量 (影响因素)

β_0 : 截距项

β_1 : 斜率 (x 每变 1, y 平均变 β_1)

ϵ : 随机误差

适用: 只有一个自变量, 且与 y 呈线性关系

2. 多元线性回归基本数学模型

数学模型：

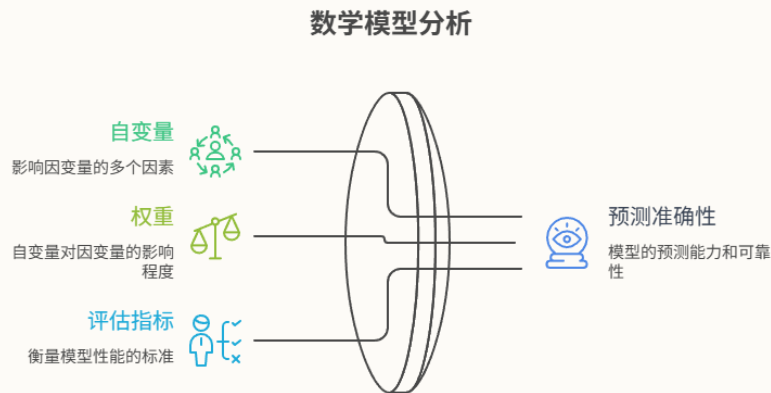
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$$

矩阵形式：

$$Y = X\beta + \varepsilon$$

含义：

- 有多个自变量共同影响 y
- β_i ：第 i 个变量的权重（影响力度）
- 评估指标： R^2 、调整 R^2 、F 检验、p 值



回归分析详细步骤



01. 问题定义与数据收集

明确预测目标（因变量）和影响因素（自变量），收集包含两者的完整历史数据。



02. 数据预处理与 EDA

清洗缺失/异常值，进行特征工程（选择、转换、缩放），利用可视化探索数据内在关系。



03. 模型选择与假设检验

根据数据分布特点选定模型（如线性回归），并严格检验模型的基本统计假设。



04. 模型训练与参数估计

划分训练集与测试集，使用训练数据通过算法（如最小二乘法）拟合出模型的最优参数。



05. 模型评估与诊断

用测试集计算 R^2 、MSE 等指标评估性能，并深入分析残差，诊断是否存在过拟合等问题。

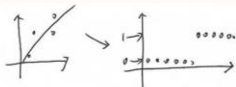


06. 模型优化与应用部署

基于评估结果调整特征或引入正则化进行优化，最终将模型部署至生产环境进行预测。

3.Logistic 回归模型

7.4 Logistic 回归



7.4.1 Logistic 模型

在回归分析中, 因变量 y 可能有两种情形: ① y 是一个定量的变量, 这时就用通常的 regress 函数对 y 进行回归; ② y 是一个定性的变量, 比如 $y = 0$ 或 1 , 这时就不能用通常的 regress 函数对 y 进行回归, 而是使用所谓的 Logistic 回归。Logistic 方法主要应用在研究某些现象发生的概率 p , 比如股票涨还是跌, 公司成功或失败的概率。除此之外, 本章还讨论概率 p 与哪些因素有关。Logistic 回归模型的基本形式为:

$$p(Y=1|x_1, x_2, \dots, x_k) = \frac{\exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}{1 + \exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}$$

其中, $\beta_0, \beta_1, \dots, \beta_k$ 为类似于多元线性回归模型中的回归系数。

$$\begin{aligned} p &= \frac{e^x}{1+e^x} \rightarrow \frac{p}{1-p} = e^x \\ \rightarrow p \ln(1-p) &= e^x \rightarrow \ln \frac{p}{1-p} = x \\ \rightarrow p &= e^x / (1+e^x) \end{aligned} \quad \begin{cases} p = \frac{e^x}{1+e^x} \\ x = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \varepsilon \end{cases}$$

第 7 章 数据回归方法 145

该式表示当变量为 $x_1, x_2, \dots, x_k$ 时, 自变量 p 为 1 的概率。对该式进行对数变换, 可得:

$$\ln \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

至此, 我们就会发现, 只要对因变量 p 按照 $\ln(p/(1-p))$ 的形式进行对数变换, 就可以将 Logistic 回归问题转化为线性回归问题, 此时就可以按照多元线性回归的方法很容易地得到回归参数。但很快又会发现, 对于定性实践, p 的取值只有 0,1, 这就导致 $\ln(p/(1-p))$ 形式失去意义。为此, 在实际应用 Logistic 模型的过程中, 常常不是直接对 p 进行回归, 而是先定义一种单调连续的概率函数 π , 令

$$\pi = p(Y=1|x_1, x_2, \dots, x_k), \quad 0 < \pi < 1$$

有了这样的定义, Logistic 模型就可变形为:

$$\ln \frac{\pi}{1-\pi} = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k, \quad 0 < \pi < 1$$

虽然形式相同, 但此时的 π 为连续函数。然后只需要对原始数据进行合理的映射处理, 就可以用线性回归方法得到回归系数, 最后再由 π 和 p 的映射关系进行反映射而得到 p 的值。下面以一个实例来更具体地介绍如何用 MATLAB 进行 Logistic 回归分析。

01. 线性预测 (Linear Prediction)

计算公式: $z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$

作用: 综合所有特征, 得到一个实数结果

02. Sigmoid 映射函数

计算公式: $p = 1 / (1 + e^{-z})$

特性: 无论 z 取何值, p 的结果始终在 (0, 1) 之间

03. 概率解释与决策边界

输出 p 代表样本属于“正类”的概率。

决策规则: $p \geq 0.5$ 预测为正类(1); 否则为负类(0)。

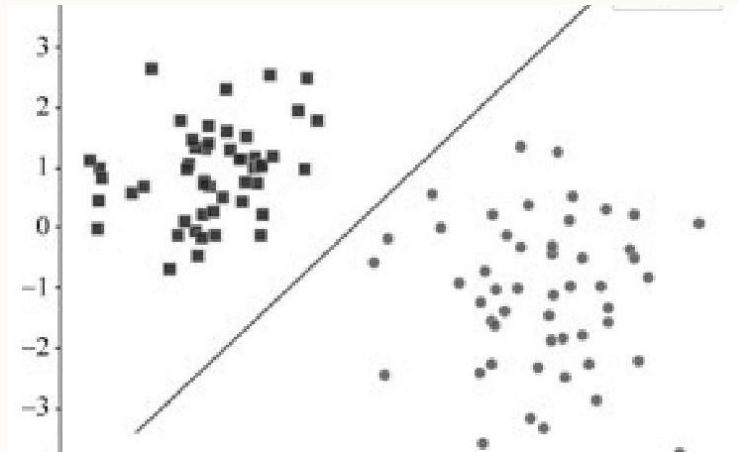
Logistic回归的决策边界

- **线性决策边界：模型的核心基石**

Logistic回归的本质是学习一个线性的决策边界。在二维平面中，它表现为一条分割直线；在三维或更高维度的特征空间中，则对应一个超平面。这是模型能够进行二元分类的几何基础。

- **分类执行：简单高效的区域判别**

该线性边界将特征空间严格划分为正负两个区域。当输入新的样本时，模型无需复杂计算，只需判断样本点落在边界的哪一侧，即可快速完成分类预测，实现高效的判别任务。



- **可视化图解析：**如右上图表所示，这是一个典型的二分类数据集。图中的斜线即为Logistic回归算法自动拟合出的线性决策边界。所有位于直线一侧的数据点被归为一类，另一侧的数据点则归为另一类，直观地展示了模型的分类逻辑。

线性回归 vs Logistic回归

👉 模型的“血缘”联系

参数求解逻辑相通，均可通过极大似然估计（MLE）等经典方法拟合模型参数。

同属广义线性模型家族，共享“线性组合”的核心骨架（ $z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$ ）。

⚖️ 本质的“任务”差异

目标天差地别：线性回归做“预测”（房价/销量），Logistic回归做“分类”（是否患病/是否点击）。

输出范围限定：线性回归输出任意实数，Logistic回归通过Sigmoid函数压缩至(0,1)区间表示概率。

📄 深度维度对比：从变量到假设

线性回归要求因变量是连续数值，且假设误差项服从正态分布，致力于拟合数据的“趋势线”；而Logistic回归的因变量是0/1的离散标签，假设数据服从二项分布，目标是寻找最佳的“决策边界”。简单来说，一个是“画直线看趋势”，一个是“划界限做判断”。

💡 总结：

如果你的需求是预测具体的数值（如明天的温度、下个月的销售额），请选择线性回归；如果你的需求是判断样本的类别归属（如是否为垃圾邮件、客户是否会流失），Logistic回归则是更合适的首选模型。

4. 白话机器学习读书笔记

回归的本质：拟合与预测

回归分析的核心是构建“**最佳拟合线**”，揭示变量间关系以预测未知。它能利用多变量提升预测精度，并比较各变量对结果的影响强度。

线性回归：寻找最优直线

线性回归旨在寻找使数据点预测误差最小的直线。模型表现为线性方程，方程系数代表各变量对结果的影响方向和强度。

模型评估：最小化预测误差

回归模型的优劣通过预测误差（预测值与实际值的差异）衡量。平均预测误差越小，代表模型的预测能力越强。

过拟合与正则化

多重共线性易导致过拟合。可通过“套索回归”或“岭回归”等正则化技术，引入惩罚项简化模型，从而提升模型泛化能力。

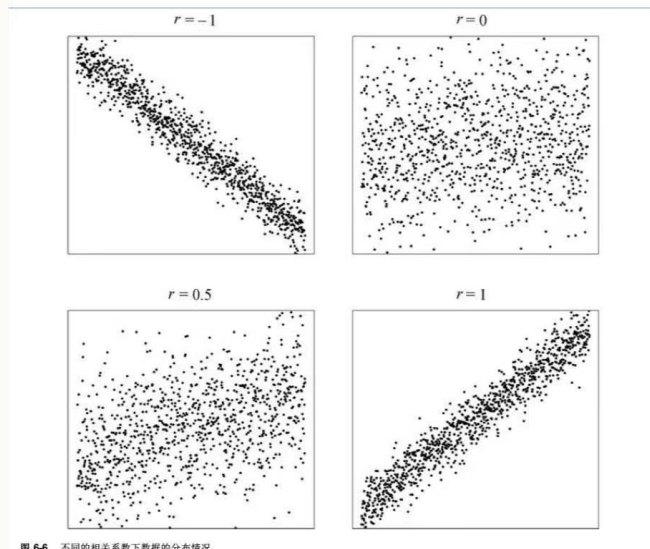


图 6-6 不同的相关系数下数据的分布情况

局限性与总结

■ 线性假设的限制

基础回归模型通常假设变量间存在线性关系，这使其难以拟合和处理现实世界中复杂的非线性数据分布。

■ 对异常值高度敏感

基于最小二乘法的回归模型，目标是最小化误差平方和。极端的异常值会显著“拉偏”拟合结果，降低模型的鲁棒性。

■ 多重共线性问题

当输入特征之间存在高度线性相关时，会导致参数估计值变得不稳定，方差显著增大，影响模型对变量影响的解释能力。

■ 相关关系 $\neq$ 因果关系

回归分析只能揭示变量之间的统计相关性，不能直接推断变量间存在因果关系。因果推断需要结合专业领域知识和实验设计。



数据挖掘与机器学习 第三次作业

分类



目录

壹

分类算法基本概念与逻辑

贰

KNN 算法详解

叁

《白话机器学习》读书笔记

一、分类算法基本概念

—

分类的定义：监督学习的核心任务之一，目标是将未知样本映射到预定义的离散类别中。

基本概念：

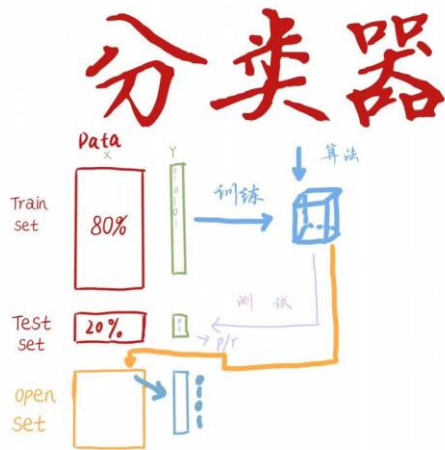
分类器：能够解释原始数据，能够预测新数据的模型；

训练集：用来训练模型的原始数据（划分出一部分）；

类标签：原始数据中已经给每个记录（对象）标记的类别；

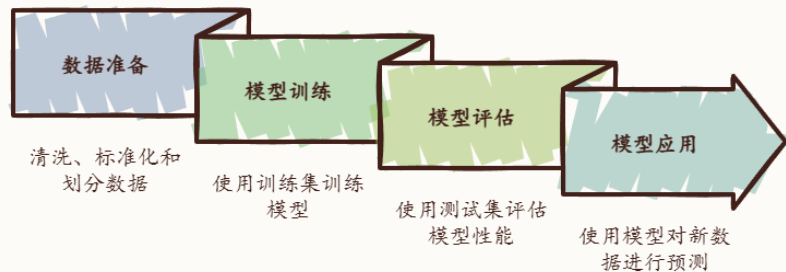
测试集：用来测试模型的原始数据（划分出的一部分）；

开放集：使用训练好的模型（分类器）进行分类工作的新数据；



一、分类算法的基本逻辑

分类算法的基本逻辑



核心目标：构建一个分类器，使其能尽可能准确地预测未知样本的类别。

通用流程：

- ① 数据准备：清洗、标准化、划分训练 / 测试集。
- ② 模型训练：用训练集让算法学习特征与标签的映射关系。
- ③ 模型评估：用测试集检验模型的分类准确率、泛化能力。
- ④ 模型应用：用训练好的模型对新数据进行分类预测。

二、KNN 算法的核心思想

—

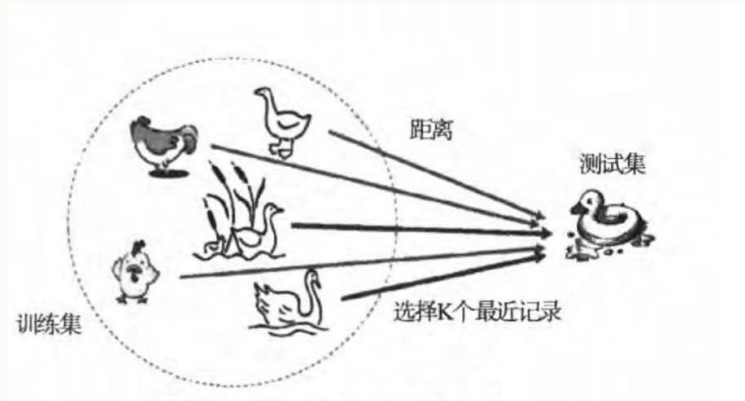
核心逻辑：“物以类聚，近朱者赤”——样本的类别由其最近的 K 个邻居的多数投票决定。

关键前提：

距离度量：定义样本间的“相似性”，常用欧氏距离。

K 值选择：决定参与投票的邻居数量，过小易过拟合，过大易欠拟合。

投票规则：多数表决法（ K 个邻居中出现次数最多的类别）。



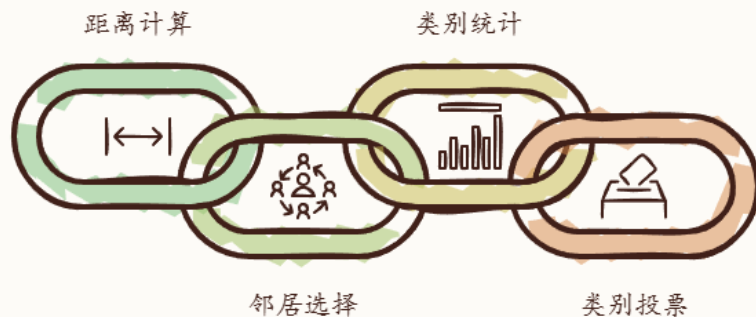
二、KNN 算法的步骤与公式

—

K-近邻分类方法：

- ① 通过计算待分类样品到每个训练样例到的距离，（度量）
- ② 取和待分类样品距离最近的K个训练样例，（邻居）
- ③ K个样品中哪个类别的训练样例占多数（统计），
- ④ 则待分类样品就属于哪个类别（投票）。

K-近邻分类方法



二、KNN 算法的优缺点

优点	缺点
简单直观，易于理解和实现	预测时需遍历所有训练样本，计算量大、速度慢
无需训练过程（懒惰学习），新数据可直接加入	对高维数据效果差（维度灾难）
对非线性数据也能较好拟合	对噪声、离群点敏感
可用于分类和回归任务	K 值和距离度量的选择会显著影响模型效果

三、读书笔记：第七章 KNN 算法

—

核心思想：基于“距离”的懒惰学习算法，核心是“近朱者赤”，无显式训练过程。

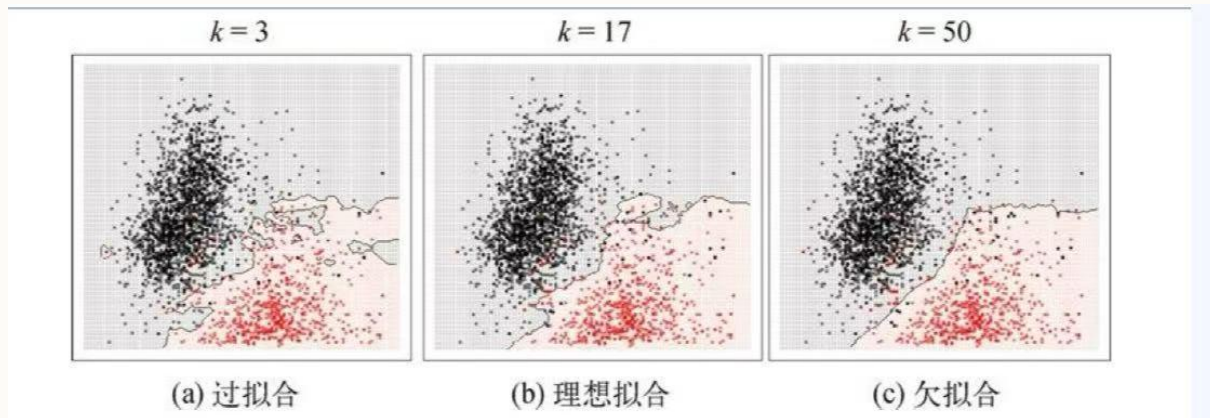
关键要素：距离度量（欧氏/曼哈顿距离）、K 值选择、投票规则。

核心特点：

优势：简单易解释，对数据分布无假设。

局限：计算成本高，不适合大数据场景，需数据标准化。

应用场景：推荐系统、图像识别、文本分类等低维、小规模数据场景。



三、读书笔记：第九章 决策树 核心思想

- 核心思想：“分而治之”，模拟人类决策过程，通过树状结构的分支规则完成分类/回归。

关键概念：

- 根节点：数据集的入口，代表最顶层的决策规则。
- 内部节点：根据特征条件分裂数据的节点。
- 叶节点：最终的分类/回归结果。
- 构建逻辑：递归地选择最优特征分裂数据，直到节点数据足够“纯净”或达到停止条件。



三、读书笔记：第九章 决策树 特点与优缺点

核心特点：模型可解释性强，无需特征标准化，能处理非线性数据。

优点：

结构清晰，决策规则直观易懂。

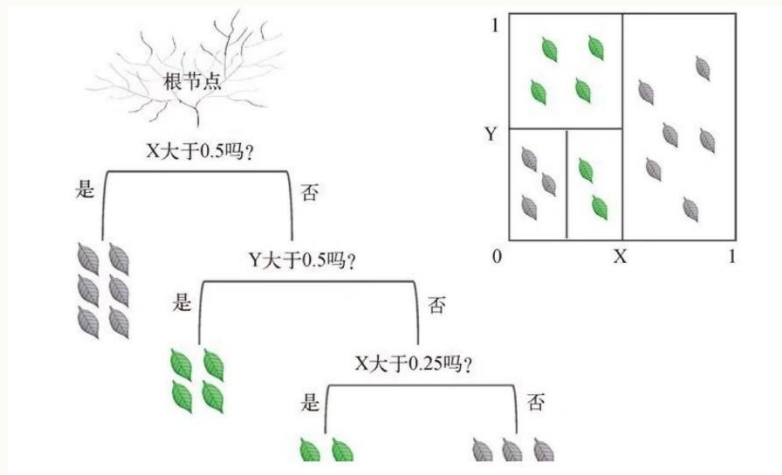
能自动处理特征交互，无需复杂预处理。

可同时用于分类和回归任务。

缺点：

容易过拟合，对训练数据的微小变化敏感。

可能偏向选择取值较多的特征，泛化能力受剪枝策略影响大。

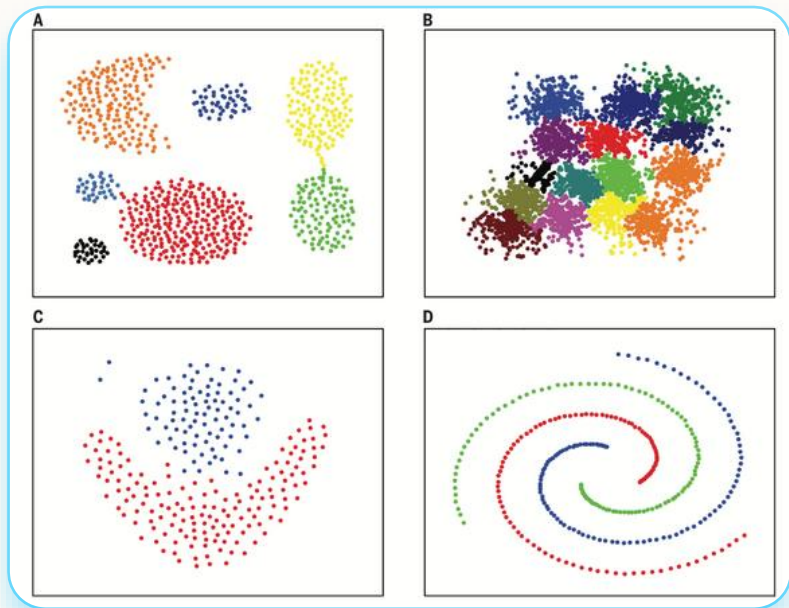




数据挖掘与机器学习 第四次作业

聚类

一、整理簇的概念



- 什么是簇？

簇是数据对象的集合，其核心在于：

簇内相似性：簇中的对象彼此非常相似。

簇间差异性：不同簇中的对象彼此非常不相似。

- 簇的形象表现在空间分布上不是确定的，而是呈各种不同的形状，在二维平面里就可以有各种不同的形状，如图所示，在多维空间里有更多的形状。

二、类度量的两种基本方法(欧式距离&余弦距离)

欧氏距离：衡量的是两个点在多维空间中的绝对距离。

核心特点：

- 数值驱动：它关注的是数值大小的差异。
- 三角不等式：满足距离度量的所有性质。
- 维度敏感：在高维空间中，由于“维度灾难”，所有点之间的距离可能会趋同，导致欧式距离失效。

余弦距离（相似度）：衡量的是两个向量在方向上的夹角。它关注的是方向的一致性，而不是长度。

核心特点：

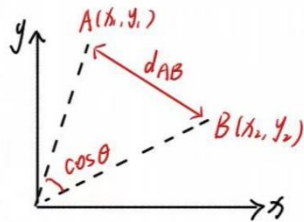
- 方向驱动：它对向量的长度（模长）不敏感。
- 值域：余弦相似度的值域在 $[-1,1]$ 之间。1 代表方向完全相同，0 代表正交，-1 代表方向完全相反。
- 归一化效果：本质上自动对向量进行了归一化处理（忽略长度）。

二维平面上点 $a(x_1, y_1)$
与 $b(x_2, y_2)$ 间的欧氏距离

$$d_{12} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

二维空间中向量 $A(x_1, y_1)$
与向量 $B(x_2, y_2)$ 的夹角余弦公式：

$$\begin{aligned}\cos\theta &= \frac{x_1x_2 + y_1y_2}{\sqrt{x_1^2 + y_1^2} \sqrt{x_2^2 + y_2^2}} \\ &= \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|}\end{aligned}$$

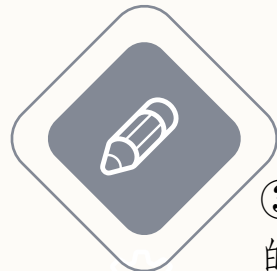


三、整理K-means算法的原理和步骤



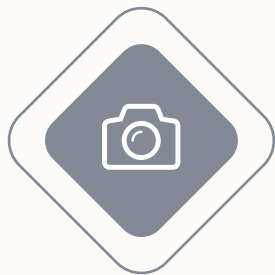
1. 初始化

① 首先随机从数据集中选取 K 个点，每个点初始地代表每个簇的聚类中心；



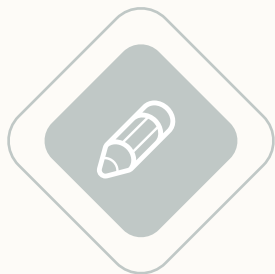
3. 更新中心

③ 接着重新计算每一簇的平均值，作为新的聚类中心；



2. 分配样本

② 然后计算剩余各个样本到聚类中心的距离，将它赋给最近的簇；



4. 收敛判断

④ 整个过程不断重复，如果相邻两次调整没有明显变化（点的归属或者聚类中心不再改变），说明数据聚类形成的簇已经收敛。

K-means算法的步骤

算法步骤:

- 1) 从 n 个数据对象中任意选择 k 个对象作为初始聚类中心;
- 2) 循环 3) 到 4) 直到每个聚类不再发生变化为止;
- 3) 根据每个聚类对象的均值 (中心对象), 计算每个对象与这些中心对象的距离; 并根据最小距离重新对相应对象进行划分;
- 4) 重新计算每个聚类的均值 (中心对象), 直到聚类中心不再变化。这种划分使得下式最小:

$$E = \sum_{j=1}^k \sum_{x_i \in \omega_j} \|x_i - m_j\|^2$$

NP难题

局部最优解

K-means 算法是很典型的基于距离的聚类算法, 采用距离作为相似性的评价指标, 即认为两个对象的距离越近, 其相似度就越大。该算法认为簇是由距离靠近的对象组成, 因此把得到紧凑且独立的簇作为最终目标。

四、整理HC算法的过程



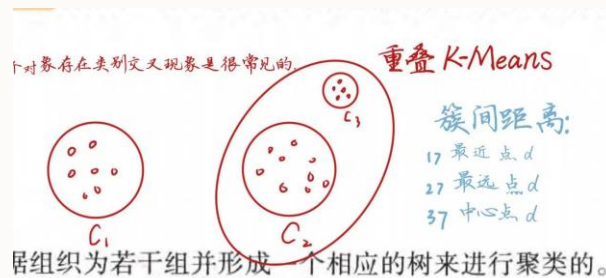
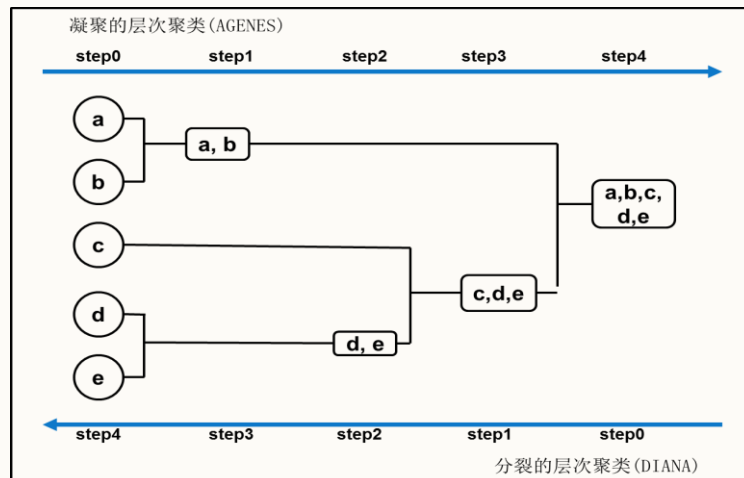
凝聚的层次聚类

- 将每个对象作为一个簇
- 计算所有簇之间的距离，找到距离最近的两个簇合并为新簇
- 重复执行合并操作，直到所有对象都在一个簇中或者达到某个终结条件要求。

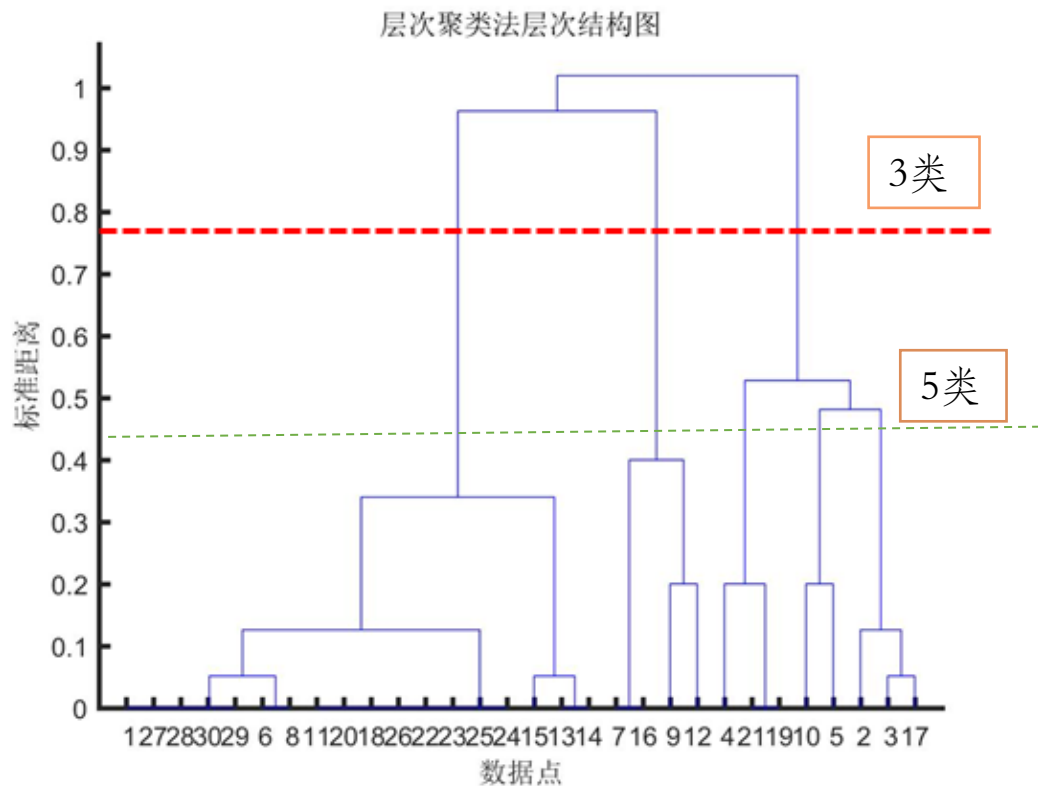


分裂的层次聚类

- 将所有对象放在一个簇中
- 递归选择当前簇将其划分为子簇
- 重复分裂直到每个对象自成一簇或达到预设簇数



五、HC聚类结果图

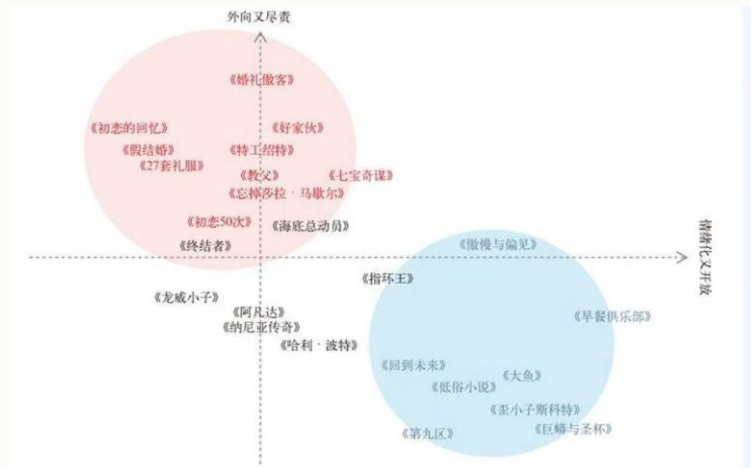


在左图中添加横线，
当这个线从上往下
平移，它与多少垂
直线相交，就分成
了几个类

六、《白话机器学习》读书笔记

- 核心理念

K-means是一种无监督学习算法，用于将相似的数据点自动分组（聚类），帮助发现数据中隐藏的顾客群体或模式。其核心假设是“相似的个体应属于同一群组”。



- 示例：影迷的性格特征

研究人员通过问卷调查获取用户在四种性格特征（外向型、尽责型、情绪型、开放型）上的得分，并将这些得分与用户在Facebook上点赞的电影关联起来。利用k均值聚类，发现了两类影迷群体：

外向且尽责的影迷：喜欢动作片和爱情片；

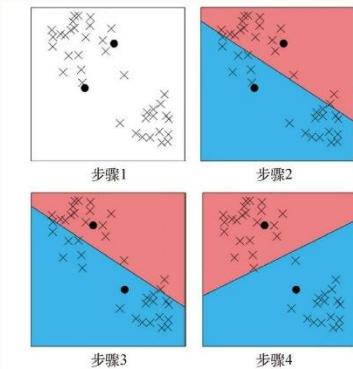
情绪化且开放的影迷：喜欢先锋艺术片和奇幻片。

《白话机器学习》读书笔记

• 关键概念

k : 表示预设的群组
(簇) 数量。

群组中心点: 每个簇的中心位置, 初始随机选择, 后通过迭代优化。



因为群组最好由密集的数据点组成, 所以通过检查群组成员与群组中心点的距离, 便可以判断该群组的有效性。不过, 由于中心点的位置最初是未知的, 因此只能先预估, 然后把数据点分配给离它们最近的中心点。

接下来, 不断调整中心点的位置。直至与真实的位置重合。而后, 根据各数据点与中心点的距离, 重新为每个数据点分配群组。如果某个数据点离原先所在群组的中心点较远, 而离邻近群组的中心点较近, 那就把它重新划入邻近的群组。

简单总结一下, 为群组确定成员的过程包含如下步骤。不管群组数量有多少, 这些步骤都适用。

步骤 1: 首先猜测每个群组的中心点。因为暂时不能确定通过猜测得到的中心点是否正确, 所以称它们为伪中心点。

步骤 2: 把每个数据点分配给最近的伪中心点。这样一来, 就得到了两个群组, 即红色群组和蓝色群组, 如图 2-3 所示。

步骤 3: 根据群组成员的分布, 调整伪中心点的位置。

步骤 4: 重复步骤 2 和步骤 3, 直至群组成员不再发生变化。

• 算法步骤

- ① 初始化中心点: 随机选择 k 个数据点作为初始的簇中心。
- ② 分配数据点: 将每个数据点分配给离它最近的中心点, 形成 k 个临时簇。
- ③ 更新中心点: 重新计算每个簇内所有点的均值, 将该均值作为新的中心点。
- ④ 重复: 重复步骤3和4, 直到中心点不再变化或达到预设迭代次数。

《白话机器学习》读书笔记

随着群组数量增加，每个群组中的成员彼此越来越相似，相邻群组之间的区别则越来越不明显。在极端情况下，每个数据点本身就是一个群组，但这种分组方式毫无意义。

显然，在决定群组数量时必须有所权衡。首先，群组数量要足够大，以便提取有意义的模式，用作商业决策参考；其次，还要足够小，能够确保各个群组之间有明显的区别。

要确定合适的群组数量，一种方法是使用陡坡图，如图 2-2 所示。

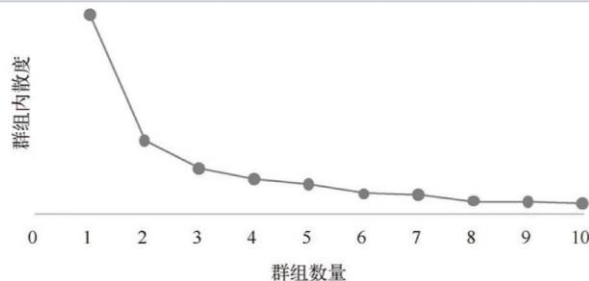


图 2-2 弯曲的陡坡图表明存在两个群组

陡坡图可以展现群组内散度随群组数量增加而降低的过程。若所有成员都属于同一个群组，则群组内散度将达到最大值。随着群组数量增加，各个群组变得更紧凑，群组成员也变得更为相似。

陡坡图曲线的拐弯处表示最佳群组数量，此处的群组内散度较为合理。从图 2-2 可以看到，当群组数量为 2 时，曲线拐弯，这两个群组对应于图 2-1 中的两个主要的电影群组。当群组数量为 3 时，曲线再次拐弯（尽管不如前一个明显），这意味着可以分出第 3 个群组，即普遍受欢迎的电影。但是，若继续增加群组数量，会导致群组变小，还会增大区分各个群组的难度。

确定好合适的群组数量之后，就该确定每个群组的成员了。

• 如何确定 k 值

书中介绍了陡坡图方法：

横轴：k 值（簇的数量）

纵轴：簇内散度（簇内点与中心点的距离之和）

随着 k 增加，散度下降，曲线拐弯处对应的 k 值即为最佳选择

《白话机器学习》读书笔记

• 局限性

形状受限：只能识别正圆形的簇，无法处理椭圆形或不规则形状。

强制划分：每个点只能属于一个簇，不能处理模糊边界或重叠簇。

对初始值敏感：不同的初始中心点可能导致不同的聚类结果。

受离群点影响：异常值会拉偏中心点位置。

2.4 局限性

尽管 均值聚类方法很有用，但是它本身存在一定的局限性。

- 每个数据点只能属于一个群组。然而，数据点可能恰好位于两个群组中间，无法通过 均值聚类方法确定它应该属于哪个群组。
- 群组被假定是正圆形的。查找距离某个群组中心点最近的数据点，这一迭代过程类似于缩小群组的半径，因此最终得到的群组在形状上类似于正圆形。假设群组的实际形状是椭圆形，那么在应用 均值聚类方法之后，位于椭圆两端的数据点可能会被划入邻近的群组，这会造成很大的问题。
- 群组被假定是离散的。 均值聚类既不允许群组重叠，也不允许它们相互嵌套。

除了 均值聚类之外，还有更可靠的聚类方法。这些聚类方法不会强制把每个数据点划入单个群组，而会计算每个数据点属于其他群组的概率，因此适合用来识别非圆形或有重叠的群组。

尽管 均值聚类存在上述局限性，但是它的优点是简单朴素。一个好的数据分析策略是，先用 均值聚类方法大致了解数据结构，再综合运用其他更高级的方法进行深入分析，这样做可以大大弥补 均值聚类方法的局限性。

2.5 小结

- 均值聚类用于把相似的数据点划入同一个群组，群组数量 必须事先指定。
- 给数据点分组时，首先把各个数据点分配到距离最近的群组中，然后调整群组中心点的位置。重复这两个步骤，直到群组中的成员不再发生变化。
- 均值聚类最适合用于正圆形、非重叠的群组。



数据挖掘与机器学习 第五次作业

关联规则

1、整理资料：有关关联选股的背景知识

常见的4种关联类型

关联选股，就是利用股票之间在**行业、题材、资金、走势等方面的关联关系**，当一只股票大涨或出现异动时，去寻找与其关联性强的其他股票，博取联动上涨的机会。

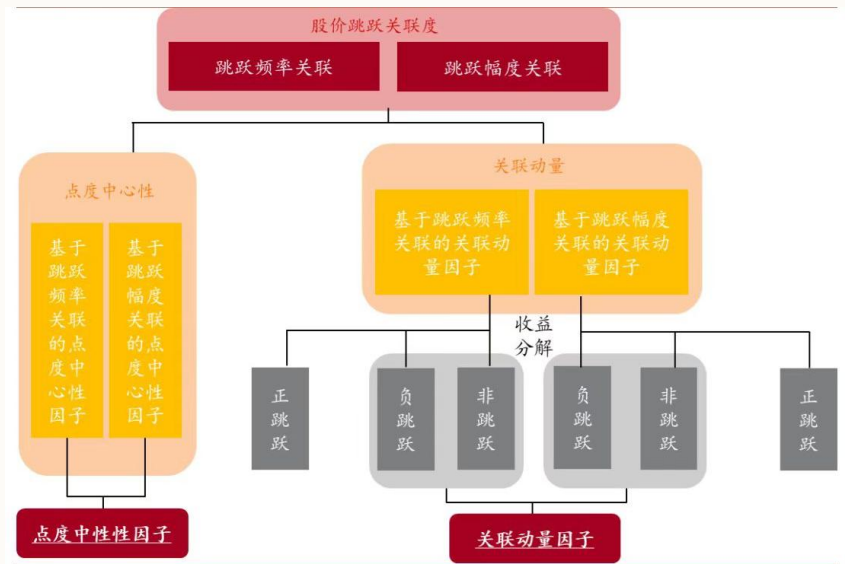
打个比方：这就像班里一个学霸突然成绩飙升，你推测和学霸关系好的同学可能也会进步，于是提前关注他们。

- ① **产业链关联：**就像宁德时代涨了，资金会去挖电池材料、设备等上下游公司。
- ② **题材概念关联：**所有和“机器人”概念沾边的股票，即使主营差异大，也会被归为一类联动。
- ③ **股东背景关联：**某牛散举牌了一只股，他持有的其他股票可能会被炒作；或同一控股股东的兄弟公司。
- ④ **技术走势关联：**两只股票长期走势高度相似，一只突破，另一只被认为有补涨潜力

1、整理资料：有关关联选股的背景知识

•**选股先选行业：**行业是公司成长的核心土壤，其景气度直接决定企业发展的天花板。处于上升周期的行业能为公司提供天然的增长红利与广阔的市场机遇，远非单个公司的个体努力所能弥补。

•**行业轮动现象：**受宏观经济周期、市场情绪及政策导向影响，不同行业会呈现阶段性的涨跌交替特征。投资者通过识别行业所处的周期位置，能更精准地把握板块轮动带来的投资机会。

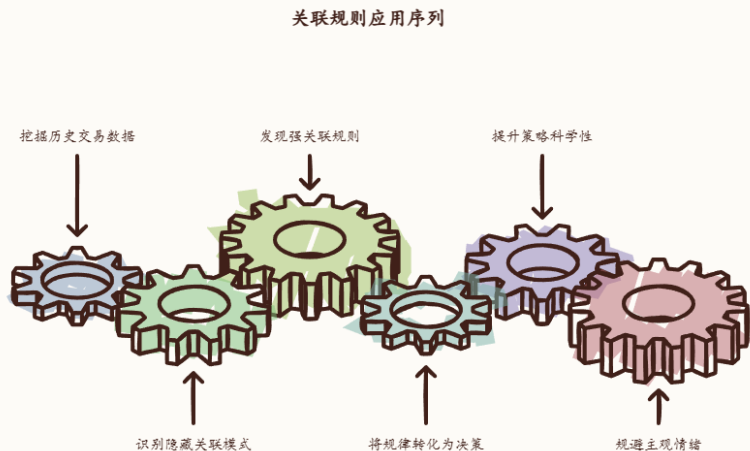


资料来源：招商证券

1、整理资料：有关关联选股的背景知识

•关联规则的核心应用：

- 利用 Apriori 等经典数据挖掘算法，深度挖掘历史交易数据中行业间的隐藏关联模式。
- 通过分析海量数据，可发现如“A行业指数上涨后，B行业在未来5个交易日内大概率上涨”这类强关联规则，打破传统经验判断的局限。
- 将数据挖掘得出的客观规律转化为投资决策依据，有效提升策略的科学性与胜率，规避主观情绪对选股的干扰。



2、分析数据（清楚说明矩阵行与列的含义）

交易日	银行	房地产	医药	科技
Day 1	1	1	0	0
Day 2	0	0	1	0
Day 3	1	1	0	1
Day 4	0	0	0	1
Day 5	1	0	1	0
Day 6	1	1	0	0
Day 7	0	0	1	1

•**矩阵维度构建：**以“交易日”为行，以“行业板块（银行、地产等）”为列，搭建基础数据矩阵框架。

•**二元数值定义：**矩阵值为0或1，行业当日上涨记为1，下跌或持平则记为0，简化复杂的市场涨跌数据。

行 (Rows) - “交易日”：

每一行代表一个独立的交易日，可以把每一行想象成一个“购物篮”。

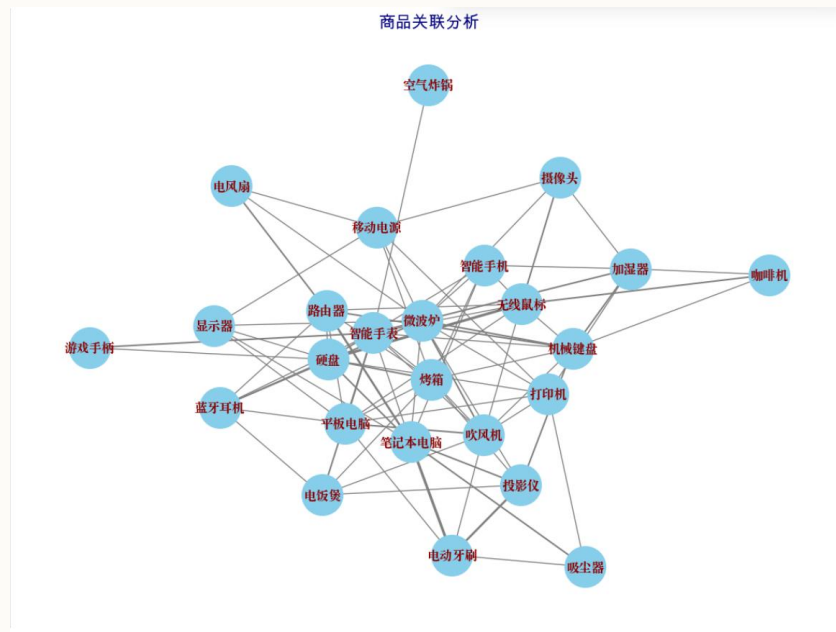
列 (Columns) - “行业板块”：

每一列代表一个特定的行业板块（如 银行，房地产），可以把每一列想象成一种“商品”。

通过这样的转换，我们就将“哪些行业在哪些日子一起上涨”的问题，转化成了经典的“购物篮分析”问题，从而可以使用 Apriori 等算法来挖掘其中的关联规则。

3、运行程序

Delimiter: , v				
	序号	商品1	商品2	商品3
1	1	蓝牙耳机	电饭煲	投影仪
2	2	打印机	烤箱	投影仪
3	3	微波炉	摄像头	无线鼠标
4	4	咖啡机	加湿器	空气炸锅
5	5	硬盘	显示器	咖啡机
6	6	笔记本电脑	路由器	电饭煲
7	7	摄像头	无线鼠标	吸尘器
8	8	电动牙刷	显示器	投影仪
9	9	笔记本电脑	摄像头	机械键盘
10	10	加湿器	机械键盘	微波炉
11	11	机械键盘	空气炸锅	智能手表
12	12	电动牙刷	投影仪	打印机
13	13	移动电源	投影仪	电动牙刷
14	14	投影仪	摄像头	咖啡机
15	15	智能手表	蓝牙耳机	机械键盘
16	16	加湿器	电动牙刷	蓝牙耳机
17	17	投影仪	移动电源	平板电脑
18	18	空气炸锅	吸尘器	移动电源



4、分析结果，思考教材分析过程的含义

Apriori 商品关联挖掘结果

- ① **数码硬件集群关联性最强**：笔记本电脑、硬盘、路由器、蓝牙耳机、平板、智能手表、智能手机形成密集关联圈，同订单高频连带购买；其中笔记本电脑 & 路由器组合支持度 0.04，全样本出现 4 次，是全数据集关联性最高组合。
- ② **小家电跨品类连带明显**：加湿器为跨品类枢纽单品，同时和咖啡机、打印机、摄像头、智能手机、智能手表等多品类高频搭配，是全场景百搭商品。
- ③ **厨电/个护相对分散**：空气炸锅、电饭煲、电动牙刷、吸尘器仅少量局部关联，跨大类组合少。

4、分析结果，思考教材分析过程的含义

分析含义

将交易日类比购物篮、行业上涨类比商品选购，用 Apriori 挖掘行业同步上涨规律；依托板块联动，在关联行业尚未拉升时提前买入，实现行业轮动选股。

第6章 关联规则方法 123

1	0	0	0	0	0	7
0	1	0	0	0	0	7
0	0	1	0	0	0	6
0	0	0	1	0	0	3
0	0	0	0	1	0	3
0	0	0	0	0	1	3
1	1	0	0	0	0	6
1	0	1	0	0	0	4
0	1	1	0	0	0	4
1	1	1	0	0	0	3

由程序的执行结果，我们看出，满足最小支持度 3 的包含 3 个行业项的项集只有一个，即：

{银行,券商,钢铁: 3/10}

这说明这 3 个行业在一定周期内（10 个交易日）具有较高（3/10）的联动可能性。再看包含 2 个行业项的项集，这里发现 2 个：

{银行,钢铁: 4/10}

{银行,券商: 4/10}

而且它们出现联动的概率是一致的，都为 4/10，所以在实践中，如果出现这 3 个行业中的一个出现涨势，那么就可以考虑从其他 2 个行业中选择代表性的股票进行买入，这样就可以在其他行业还没上涨时，提前埋伏进去，以此获得较高的收益。

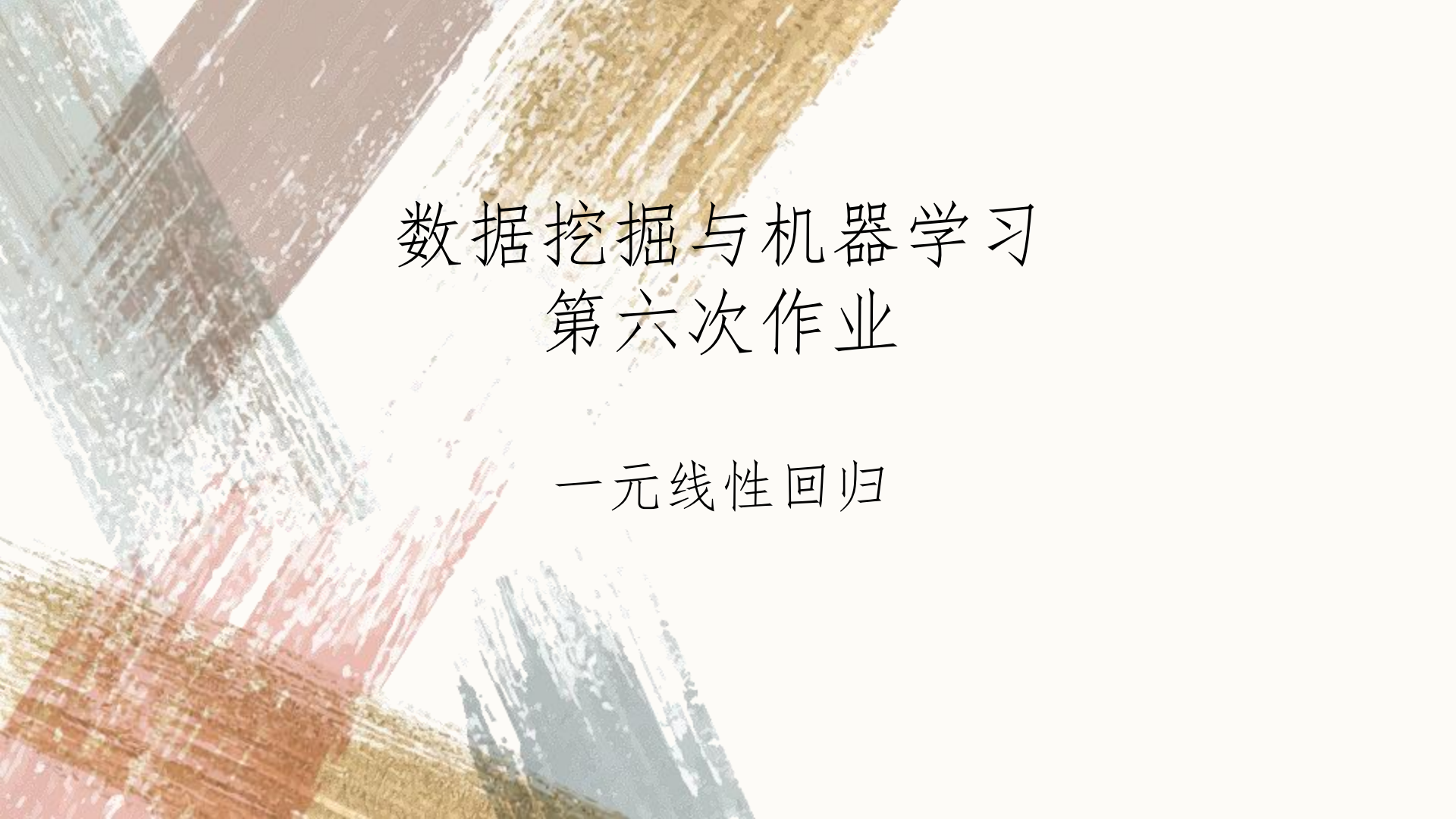
以上就是行业关联选股法的基本思想、实现方式和操作步骤，当然这里介绍的案例只是一个原型，在具体操作中可以更灵活。

错误②：Apriori 数学逻辑错误

原文内容：二项组合 {银行，券商}=4/10、{银行，钢铁}=4/10；三项 {银行，券商，钢铁}=3/10

频繁 3 项集的全部 2 项子集，出现次数一定 ≥ 3 项集次数，三项同涨的 3 个交易日，已经被计入两组二项集的 4 次共现里；

错误① 3 是支持度计数（出现天数），3/10 才是支持度



数据挖掘与机器学习 第六次作业

一元线性回归

1、为什么X必须是列向量？变成行向量会怎么样呢？

为什么 X 必须是列向量？——
从“特征矩阵”的定义说起

在机器学习中，我们处理的数据通常被组织成一个特征矩阵 (Feature Matrix)。这个矩阵有明确的规定：

每一行 (Row)：代表一个独立的样本 (Sample)。
在我们的例子中，每一行就是一个学生的记录。

每一列 (Column)：代表一个描述样本的特征 (Feature)。
在我们的例子中，“学习时间”就是一个特征。如果还有“复习时长”、“课前预习时间”等，它们就会成为另外的列。

框架统一输入标准：X 统一格式：[样本数量, 特征数量]

第一维 = 样本条数，第二维 = 特征个数；
多特征时自动拓展为 (n, k) (k 个特征)，
单特征也要遵守格式，不能丢特征维度。

变成行向量会怎么样呢？

一维数组 (25,) 歧义：sklearn 无法区分是「**25 样本 × 1 特征**」还是「**1 样本 × 25 特征**」，拟合 `LinearRegression.fit(X, y)` 会触发维度报错。

2、 如何显示生成的“熟模型”？ $y = b_0 + b_1 * x$, b_0 和 b_1 怎么获得？

熟模型定义

执行`regressor.fit(X_train, Y_train)`
完成训练后的 **LinearRegression 实例**,
已经通过最小二乘法拟合出线性关系式:

$$y = b_0 + b_1 x$$

b_0 : 模型截距（常量项）

b_1 : 自变量 x 的回归系数（斜率）

提取 b_0 、 b_1

- 2 b_0 和 b_1 怎么生成

```
[31]: b1 = regressor.coef_ # 系数  
      b1  
[31]: array([9.91065648])  
[32]: b0 = regressor.intercept_ #截距 y = 2 + 9.9x  
      b0  
[32]: 2.018160041434669
```

`regressor.intercept_`: sklearn 固定属性,
返回**一元线性的截距数值**, 本案例可算出具
体常数

`regressor.coef_`: 返回**特征系数数组**, 单
特征场景下数组仅 1 个元素, 即 b_1

3、 如何评价结果的好坏？（代码） R^2 的结果是什么？

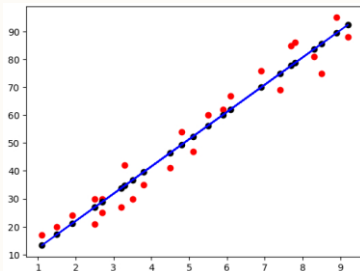
- 3 评价结果的好坏 R^2

```
[33]: R2_train = regressor.score(X_train,Y_train)
      R2_train
```

```
[33]: 0.9515510725211553
```

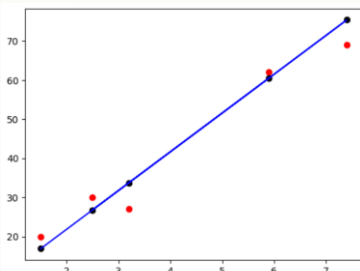
```
[34]: R2_test = regressor.score(X_test,Y_test)
      R2_test
```

```
[34]: 0.9454906892105355
```



■ 核心元素解读

● **红色散点：**模型的训练样本，是算法学习的基础数据源，反映原始数据的分布特征。



● **蓝色线条/散点：**蓝色直线是模型学到的规律；蓝色散点是测试集真实值，与黑色预测点的重合度越高，模型效果越优。

代码解读：score() 方法会自动使用模型对X_test进行预测，并将预测结果与真实的y_test进行比较，计算出 R^2 分数。

计算出的 R^2 分数非常高（约为 0.95），这表明**学习时间这个特征能够解释约 95% 的分数变异**，说明我们的线性回归模型拟合效果非常好。

4 、调整训练/测试的比例，对结果 (R^2) 有什么影响？

====不同测试集比例 R^2 结果====

test_size=0.1, $R^2=-5.2106$

test_size=0.2, $R^2=0.9692$

test_size=0.3, $R^2=0.9679$

test_size=0.5, $R^2=0.9538$

核心参数：test_size 的作用

train_test_split 函数中的 test_size 参数直接决定了数据集的划分比例，它是平衡模型训练充分性与评估可靠性的关键开关，本质上是在“学习数据量”和“评估可信度”之间做权衡。

两种极端比例的优劣势分析

训练集比例过大 (test_size=0.1)：优点是模型能学习到更丰富的数据特征，拟合潜力更高；但测试集数据量过少，评估结果偶然性大，无法真实反映模型的泛化能力，可能造成对模型效果的误判。

测试集比例过大 (test_size=0.5)：优点是评估结果更稳定、更具参考性；但留给模型学习的样本大幅减少，容易导致模型欠拟合，无法捕捉数据的核心规律，模型复杂度也会受到限制。

行业通用的经验法则

在绝大多数机器学习场景中，推荐将 test_size 设置在 0.2 到 0.3 之间。这个区间既能保证模型拥有足够的训练样本进行学习，又能提供充足的测试样本对泛化能力进行有效验证，是兼顾训练效果与评估可信度的最优平衡点。



数据挖掘与机器学习 第七次作业

多元线性回归

目录

壹

数据集特征分析与可视化

貳

One-Hot 编码过程详解

叁

模型评估： R^2 的含义

肆

系数、截距分析与最终公式

伍

思考问题

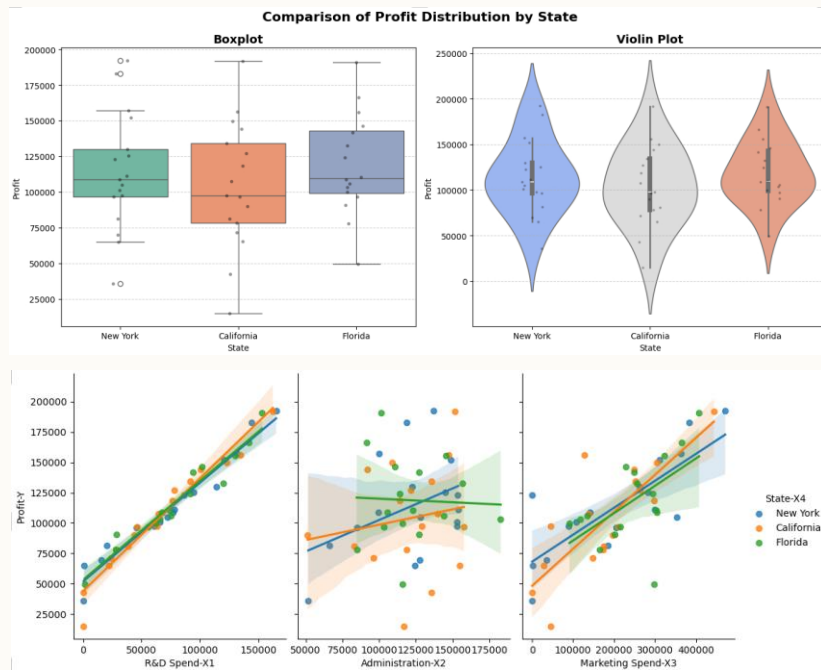
1. 数据集特征分析与可视化

数据集属性

R&D Spend	研发支出
Administration	管理支出
Marketing Spend	营销支出
State	城市
Profit	利润

State是分类变量，其取值为New York, California, Florida。

可视化



2. One-Hot 编码过程详解

核心内容

目的：将无序的分类变量（如城市）转换为机器学习模型可以理解的数值形式，同时避免引入不存在的顺序关系。

过程

- ① 使用 LabelEncoder 将 State 列转为数字 (0, 1, 2)
- ② 用 OneHotEncoder + ColumnTransformer 对第3列进行独热编码
- ③ 生成虚拟变量列（3列）
- ④ 删除第一列以避免虚拟变量陷阱
- ⑤ 将编码后的结果与原数值特征合并

```
[ ]: from sklearn.preprocessing import LabelEncoder, OneHotEncoder
      from sklearn.compose import ColumnTransformer
      labelencoder_X = LabelEncoder() # 先用LabelEncoder把分类变量标签化，就是用数字标号代替文字
      X[:, 3] = labelencoder_X.fit_transform(X[:, 3])

[ ]: X[0:10]

[ ]: array([[165349.2, 136897.8, 471784.1, 2],
            [162597.7, 151377.59, 443898.53, 0],
            [153441.51, 101145.55, 407934.54, 1],
            [144372.41, 118671.85, 383199.62, 2],
            [142107.34, 91391.77, 366168.42, 1],
            [131876.9, 99814.71, 362861.36, 2],
            [134615.46, 147198.87, 127716.82, 0],
            [130298.13, 145530.06, 323876.68, 1],
            [120542.52, 148718.95, 311613.29, 2],
            [123334.88, 108679.17, 304981.62, 0]], dtype=object)

[ ]: ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [3])], remainder='passthrough')
      X = np.array(ct.fit_transform(X))
      ***由于现在的onhotencoder会默认对整个数据集进行处理，并且没有了categorical_features，所以利用ColumnTran
      ***其实也可以通过pandas中的get_dummies()来进行onehot编码***
```

3. 模型评估：R² 的含义

定义：R²（决定系数）是衡量多元线性回归模型拟合优度的指标。

取值范围：0 ≤ R² ≤ 1。

含义：表示因变量的变异中，可以被自变量（包括数值变量和编码后的分类变量）解释的比例。

解读：

R² = 1：模型完美解释了所有数据变异。

R² = 0：模型完全无法解释数据变异。

越接近 1，模型拟合效果越好。

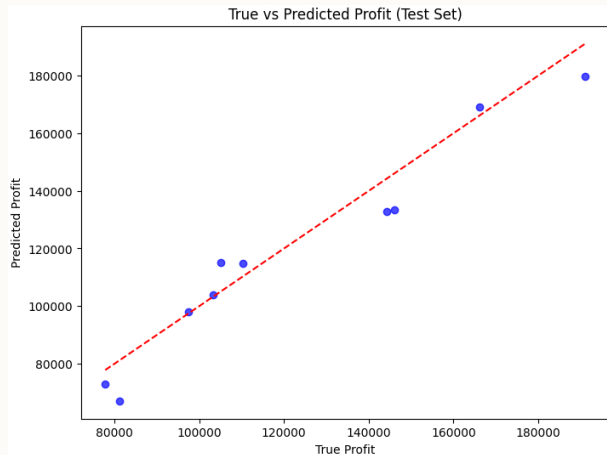
代码：regressor.score(X_test, y_test)

第4步 评估系数R²

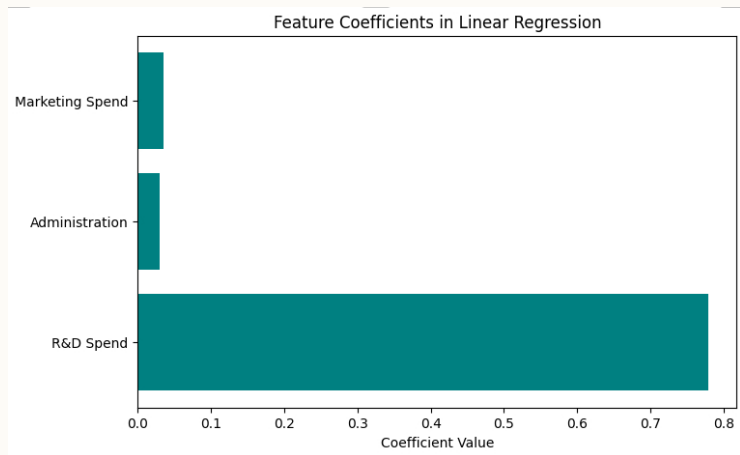
```
[59]: R2 = regressor.score(X,Y)  
R2
```

```
[59]: 0.9490884301964866
```

含义：模型能解释 94.91% 的利润变异



4. 系数、截距分析与最终公式



获取参数：

系数：`regressor.coef_`，返回一个数组，每个元素对应一个自变量的权重。

截距：`regressor.intercept_`，返回模型的常数项。

截距 $b_0 = 42989.01$

系数（权重）：

R&D Spend: 0.7788

Administration: 0.0294

Marketing Spend: 0.0347

数学公式：

$$\text{Profit} = 42989.01 + 0.7788 \times \text{R\&D} + 0.0294 \times \text{Admin} + 0.0347 \times \text{Marketing}$$

5.分类变量的影响、Label Encoding 的合理性

分类变量的影响

引入额外信息：分类变量（如城市）可以捕捉因地理位置、政策、市场环境等因素带来的差异，从而提高模型的解释力和预测准确性。

模型复杂度增加：每个分类变量会增加（类别数 - 1）个新的特征，可能导致模型变得更复杂。

需要正确编码：必须使用 One-Hot Encoding 等方法正确处理，否则会引入错误的数学关系。

Label Encoding 的合理性

Label Encoding：直接将分类变量转换为 0, 1, 2 等数字。

不合理：

引入顺序关系：将New York=0, California=1, Florida=2 会让模型错误地认为存在New York < California < Florida的顺序关系，而实际上城市之间是平等的。

数值误导：模型可能会认为Florida的影响是New York的两倍，这显然是不合理的。

结论：对于无序的分类变量，绝对不能直接使用 Label Encoding，必须使用 One-Hot Encoding。



数据挖掘与机器学习 第八次作业

KNN

1. 加入性别因素（进行one-hot编码）

```
[101]: # onehot编码

[103]: from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.compose import ColumnTransformer
labelencoder_X = LabelEncoder() # 先用LabelEncoder把分类变量标签化，就是用数字标号代替文字
X[:, 0] = labelencoder_X.fit_transform(X[:, 0])
ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0])], remainder='passthrough')
X = np.array(ct.fit_transform(X))

[105]: #from sklearn.preprocessing import LabelEncoder, OneHotEncoder
#labelencoder = LabelEncoder()
#X[:, 0] = labelencoder.fit_transform(X[:, 0])
#X[0:5]
#onehotencoder = OneHotEncoder(categorical_features=[0])
#X = onehotencoder.fit_transform(X).toarray()
#X[0:5]
```

- LabelEncoder: 把文字型的分类变量转成数字。比如 Male会变成0, Female变成1（或者反过来，取决于数据出现顺序）。
- 但这里有个问题：直接用0和1表示性别，会被模型误认为是“有序的数值”，而性别是没有顺序的（0和1之间没有大小关系），所以需要进一步做独热编码。

- ColumnTransformer: 用来对不同列应用不同的预处理方法
- OneHotEncoder(): 把分类变量转成“独热向量”，比如原来的 Gender 列（0/1），经过独热编码后会变成两列：
 - 当性别是 Male（0）：
[1, 0]
 - 当性别是 Female（1）：
[0, 1]

1. 加入性别因素（进行one-hot编码）

```
[117]: # 训练

[119]: from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
classifier.fit(X_train, y_train)

[119]: ▾ KNeighborsClassifier ⓘ ⓘ
KNeighborsClassifier()

[121]: # 预测

[123]: y_pred = classifier.predict(X_test)

[125]: from sklearn.metrics import confusion_matrix

[127]: cm = confusion_matrix(y_test, y_pred)
cm

[127]: array([[55,  3],
          [ 1, 21]], dtype=int64)

[129]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
accuracy

[129]: 0.95
```

- 加入性别特征并做独热编码后，模型准确率仍为0.95，无明显变化。
- 该数据集中年龄、薪资对购买行为的预测作用极强，性别为弱相关特征，对模型性能没有显著影响。

2.1 比较KNN与Logistic在这个数据案例上的结果差异

生成混淆矩阵

```
[31]: from sklearn.metrics import confusion_matrix
      cm = confusion_matrix(y_test, y_pred)

[33]: [y_test, y_pred]

[33]: array([[0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
            0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
            1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,
            0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1], dtype=int64),
      array([[0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
            0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
            1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
            0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1], dtype=int64)]

[35]: cm

[35]: array([[ 57,  1],
            [  5, 17]], dtype=int64)

[37]: p = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[1,1]+cm[0,1]+cm[1,0])
      p
[37]: 0.925
```

Logistic

KNN 的**非线性拟合**能力能更好地捕捉年龄、薪资与购买行为的局部规律，而 Logistic 回归的线性边界存在少量误判。

- Logistic 回归准确率: 0.925
- KNN (K=5) 准确率: 0.95
- KNN 的准确率略高于

Logistic 回归，
两者都表现不错，
但 KNN 的预测误差更少。

```
[117]: # 训练

[119]: from sklearn.neighbors import KNeighborsClassifier
      classifier = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
      classifier.fit(X_train, y_train)

[119]: KNeighborsClassifier()

[121]: # 预测

[123]: y_pred = classifier.predict(X_test)

[125]: from sklearn.metrics import confusion_matrix

[127]: cm = confusion_matrix(y_test, y_pred)
      cm

[127]: array([[55,  3],
            [ 1, 21]], dtype=int64)

[129]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
      accuracy
[129]: 0.95
```

KNN

2.2 训练集的比例，如何影响结果

```
[71]: KNeighborsClassifier
      KNeighborsClassifier()

[73]: # 预测

[75]: y_pred = classifier.predict(X_test)

[77]: from sklearn.metrics import confusion_matrix

[79]: cm = confusion_matrix(y_test,y_pred)
      cm

[79]: array([[92,  9],
           [ 6, 53]], dtype=int64)

[81]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
      accuracy

[81]: 0.90625
```

```
[31]: KNeighborsClassifier
      KNeighborsClassifier()

[33]: # 预测

[35]: y_pred = classifier.predict(X_test)

[37]: from sklearn.metrics import confusion_matrix

[39]: cm = confusion_matrix(y_test,y_pred)
      cm

[39]: array([[30,  2],
           [ 1,  7]], dtype=int64)

[41]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
      accuracy

[41]: 0.925
```

调整 train_size (左图为0.6, 右图为0.9)

规律: 训练集太少容易欠拟合, 训练集太大测试集太少评估不准, 一般 0.7-0.8 是比较常见的合适选择。

3 超参数K如何影响结果

```
[99]: cm
[99]: array([[94, 7],
           [ 9, 50]], dtype=int64)
```

- 关于混淆矩阵，更加详细的内容，我特别建议大家参考下面的博客，这样可以了解的清楚一点
- 博客地址: <https://www.jianshu.com/p/27228ad417d4>

```
[102]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) # 用对角线上的和，除以总数，就是准确率。准确率93%，还真不错呢！
```

```
[104]: accuracy
```

```
[104]: 0.9
```

K=1

```
[122]: cm
[122]: array([[94, 7],
           [ 7, 52]], dtype=int64)
```

- 关于混淆矩阵，更加详细的内容，我特别建议大家参考下面的博客，这样可以了解的清楚一点
- 博客地址: <https://www.jianshu.com/p/27228ad417d4>

```
[125]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) # 用对角线上的和，除以总数，就是准确率。准确率93%，还真不错呢！
```

```
[127]: accuracy
```

```
[127]: 0.9125
```

K=3

```
[145]: cm
[145]: array([[93, 8],
           [ 7, 52]], dtype=int64)
```

- 关于混淆矩阵，更加详细的内容，我特别建议大家参考下面的博客，这样可以了解的清楚一点
- 博客地址: <https://www.jianshu.com/p/27228ad417d4>

```
[148]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) # 用对角线上的和，除以总数，就是准确率。准确率93%，还真不错呢！
```

```
[150]: accuracy
```

```
[150]: 0.90625
```

K=7

```
[168]: cm
[168]: array([[93, 8],
           [ 7, 52]], dtype=int64)
```

- 关于混淆矩阵，更加详细的内容，我特别建议大家参考下面的博客，这样可以了解的清楚一点
- 博客地址: <https://www.jianshu.com/p/27228ad417d4>

```
[171]: accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) # 用对角线上的和，除以总数，就是准确率。准确率93%，还真不错呢！
```

```
[173]: accuracy
```

```
[173]: 0.90625
```

K=9

K 太小容易过拟合 (太敏感, 受噪声影响大)

K 太大容易欠拟合 (太平滑, 忽略数据细节)

4) 出现过拟合的情况

1.

什么是过拟合？

模型在训练集上表现极佳，但在新的、未见过的测试集上表现显著下降的现象。简单来说，模型“死记硬背”了训练数据，而没有学到真正的规律。

2.

过拟合的典型表现

准确率差异大：训练集准确率很高，测试集准确率明显偏低。

决策边界复杂：模型的决策边界过于曲折，试图捕捉每一个训练样本，甚至包括噪声点。

泛化能力差：模型对新数据的预测能力不稳定，抗干扰能力弱。

3.

何时会出现过拟合？

K 值过小（最典型）：

当 $K=1$ 时，KNN 模型会完全记住每一个训练样本，在训练集上准确率极高，但在测试集上表现很差，这是典型的过拟合。K 值越小，模型越复杂，越容易过拟合。

训练集比例过大：

如果训练集占比过高（如 90%），测试集数据过少，模型会过度适应训练数据的分布，导致泛化能力下降。

特征维度过多：

加入大量与目标变量无关的特征，会让模型学习到数据中的随机噪声，而非真正的模式，从而引发过拟合。

3. 对可视化图进行修改

- **颜色区分性别**：使用蓝色系代表男性，红粉色系代表女性。
- **形状区分购买状态**：使用圆形表示未购买，三角形表示已购买。
- **透明度与大小**：通过调整点的透明度和大小，避免数据点重叠，使可视化结果更清晰。

```
# 4. 划分训练集、特征缩放 (保持原有逻辑)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# 5. 训练模型
classifier = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
classifier.fit(X_train, y_train)

# 6. 预测与评估
y_pred = classifier.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
accuracy = (cm[0,0] + cm[1,1]) / cm.sum()
print(f"模型准确率: {accuracy:.2f}")

# 7. 可视化 (带性别区分)
gender_label = np.argmax(X[:, :2], axis=1) # 取独热编码的性别列
age = X[:, 2]
salary = X[:, 3]
purchased = y

plt.figure(figsize=(10, 6), dpi=100)
# 男性-未购买
plt.scatter(age[(gender_label==0) & (purchased==0)], salary[(gender_label==0) & (purchased==0)],
            c='steelblue', marker='o', s=30, alpha=0.6, label='Male - No Purchase')
# 男性-已购买
plt.scatter(age[(gender_label==0) & (purchased==1)], salary[(gender_label==0) & (purchased==1)],
            c='darkblue', marker='^', s=50, alpha=0.8, label='Male - Purchased')
# 女性-未购买
plt.scatter(age[(gender_label==1) & (purchased==0)], salary[(gender_label==1) & (purchased==0)],
            c='lightcoral', marker='o', s=30, alpha=0.6, label='Female - No Purchase')
# 女性-已购买
plt.scatter(age[(gender_label==1) & (purchased==1)], salary[(gender_label==1) & (purchased==1)],
            c='darkred', marker='^', s=50, alpha=0.8, label='Female - Purchased')

plt.xlabel('Age (Scaled)', fontsize=12)
plt.ylabel('Estimated Salary (Scaled)', fontsize=12)
plt.title('Age vs. Salary with Gender & Purchase Status', fontsize=14)
plt.legend(loc='upper right', fontsize=10)
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```

3. 对可视化图进行修改

- 原版的二维可视化是用年龄和薪资画散点图，用颜色区分是否购买
- 加入性别后，可以：用不同的颜色区分性别（比如蓝色、粉色；或者用不同的形状区分比如圆圈、方块）





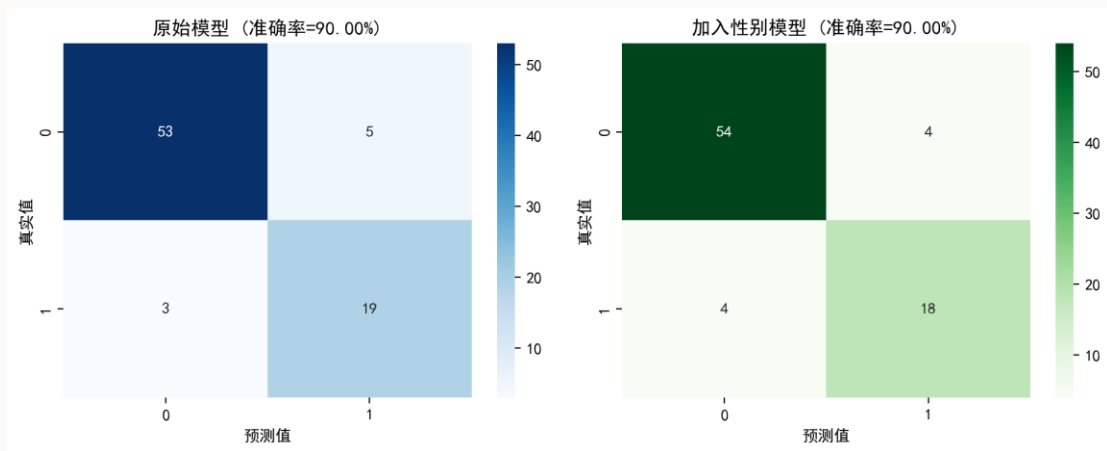
数据挖掘与机器学习 第九次作业

决策树

1、加入性别因素的影响分析

```
# ===== 4. 加入性别因素 =====  
# 对Gender进行OneHot编码  
X_gender = dataset.iloc[:, [1, 2, 3]].copy()  
# 将Gender转为数值型  
X_gender['Gender'] = X_gender['Gender'].map({'Male': 0, 'Female': 1})  
X_gender = X_gender.values  
y = dataset.iloc[:, 4].values
```

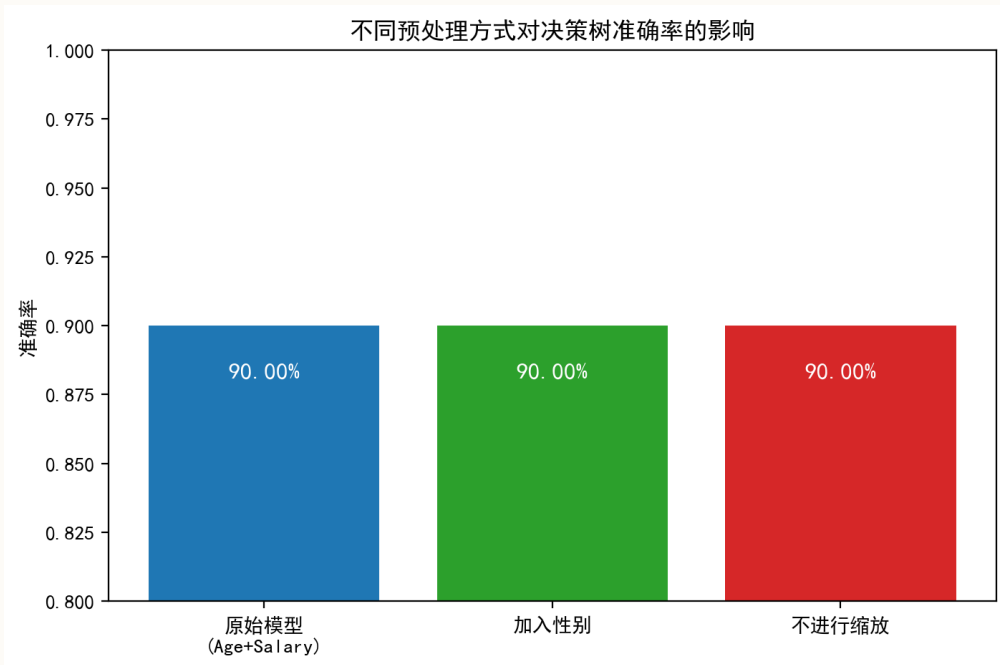
对性别进行了独热编码



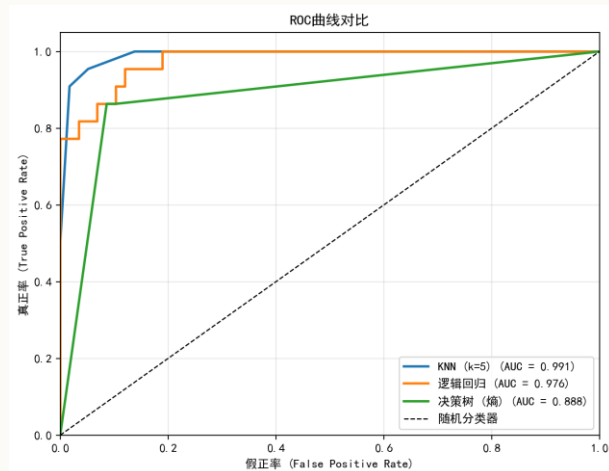
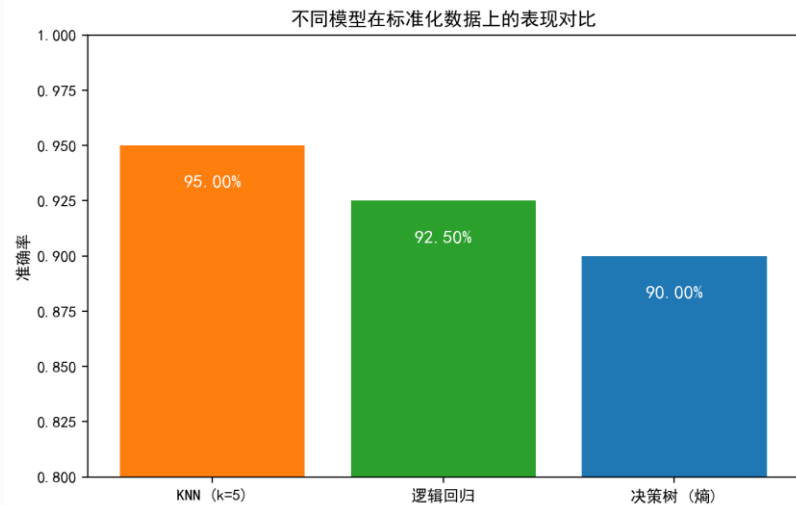
- 准确率无明显变化
→ 性别与购买决策无明显线性或非线性关系
- 可能原因：该产品（如奢侈品、科技产品）对男女吸引力均衡

2. 不进行特征缩放，预测的准确变化

- 准确率几乎相同
- 树的形状可能略有不同
(例如选择 $\text{Salary} > 50000$ vs $\text{Salary} > 0.5$ 标准化后)，但决策边界等效
- 结论：决策树对特征缩放鲁棒，可直接使用原始数据，简化生产部署



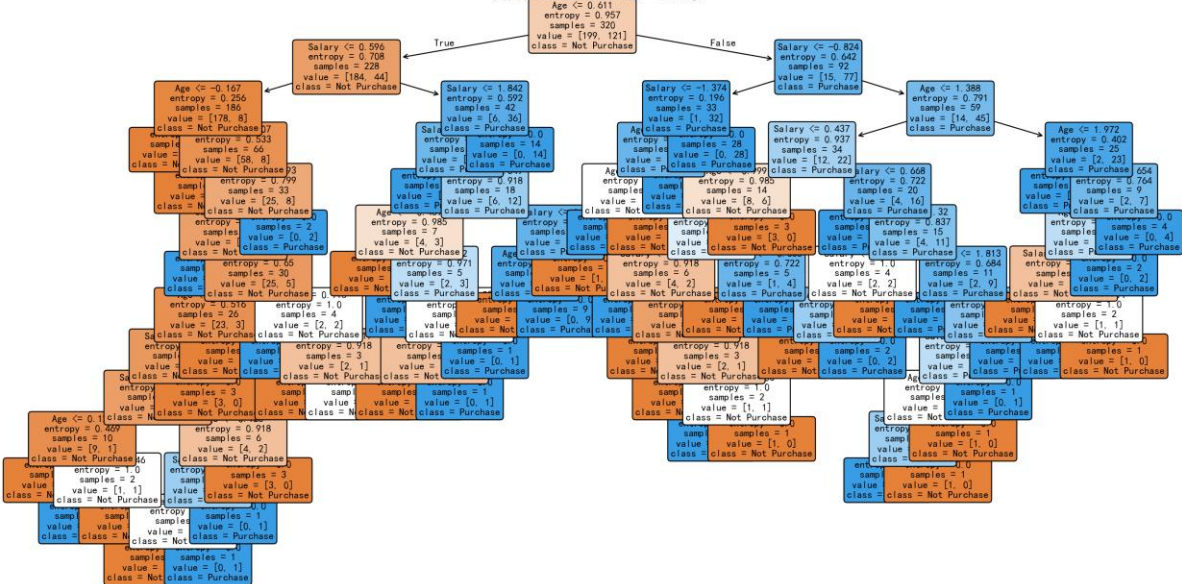
3. 比较KNN、Logistic、决策树在该数据集上的表现



- KNN: 适合小数据集、非线性边界, 但预测时需遍历所有训练样本, 且无规则输出
- 逻辑回归: 输出概率, 适合风险评分, 但特征需线性可分
- 决策树: 可生成if-then规则, 便于业务理解 (如 “**年龄<30且薪资>50k** → **购买概率高**”)

4. 决策树输出

决策树结构 (原始模型: Age + Salary)



- 根节点：选择 Age ≤ 37.5 作为第一层分裂特征
- 左分支（年龄小）：进一步考察 Salary $\leq 95000 \rightarrow$ 最终多数为“不购买”
- 右分支（年龄大）：多数为“购买”
- 叶子节点纯度：颜色越深代表类别越纯（蓝色=购买，橙色=不购买）

核心发现

性别特征对本次数据无增益，可简化数据采集

特征缩放对决策树不必要，降低工程复杂度

决策树在准确率、可解释性、速度上表现均衡，适合作为基线模型

模型对比表明：没有万能模型，需根据业务需求选择



数据挖掘与机器学习 第十次作业

K-means 聚类

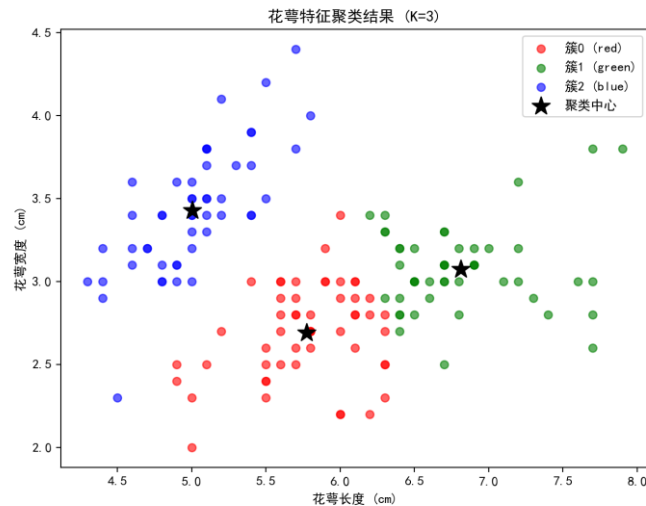
1、使用花萼特征聚类

选取特征: `iris.data[:, 0:2]` (花萼长度、花萼宽度)

K-means参数: `n_clusters=3`, `random_state=0`,
`n_init=10`

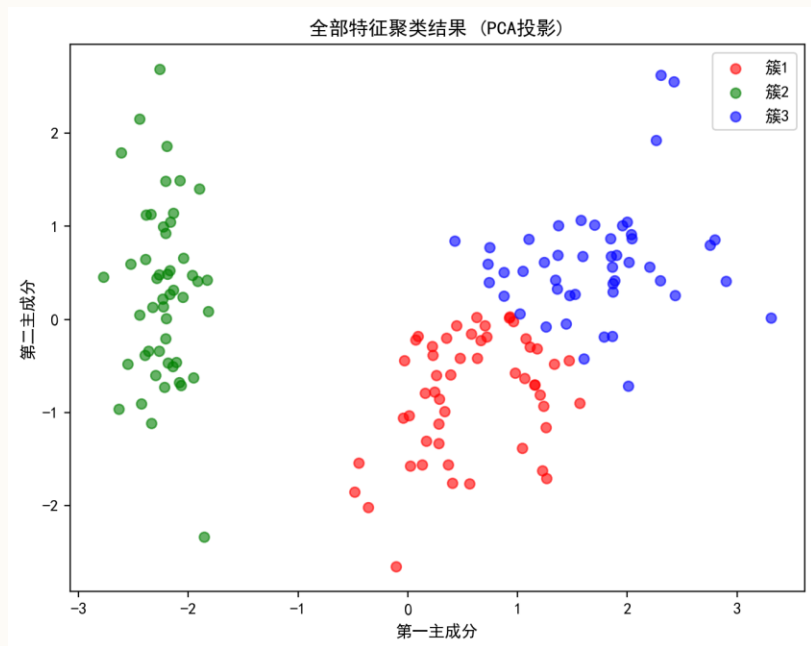
算法: K-means (欧氏距离)

```
X_sepal = X[:, :2]
kmeans_sepal = KMeans(n_clusters=3, random_state=0, n_init=10)
y_sepal = kmeans_sepal.fit_predict(X_sepal)
centers_sepal = kmeans_sepal.cluster_centers_
```



- 蓝色簇花萼最短但最宽，与其他两簇明显分离。
- 红色簇和绿色簇在花萼长度5.5~6.5、宽度2.6~3.2区域存在重叠，区分度不足。
- 轮廓系数约为0.40 (低于全特征的0.55)，说明仅用花萼特征聚类效果一般。

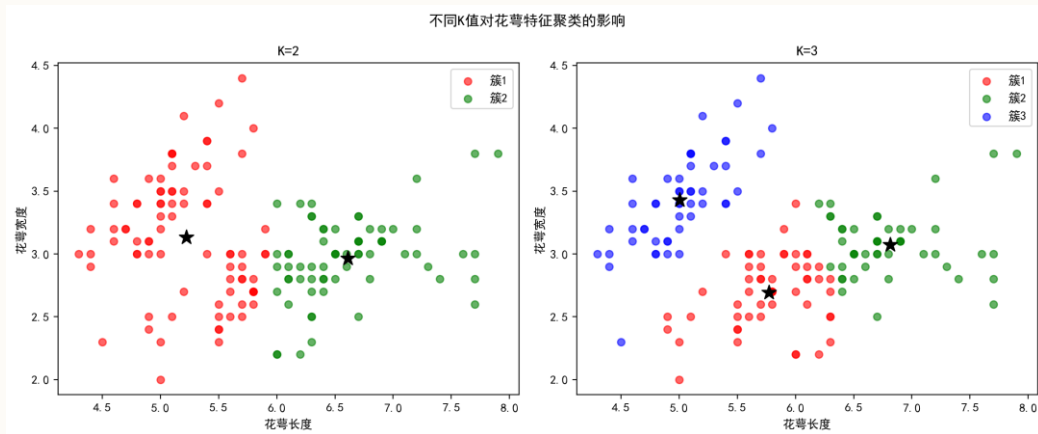
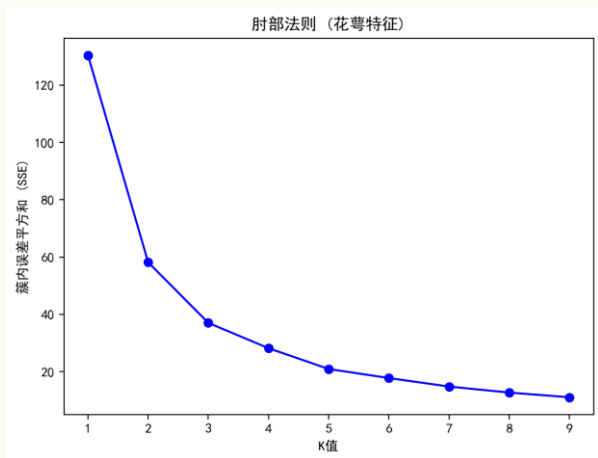
2、使用全部四个特征聚类



三个簇在PCA投影中几乎完全分离，说明花瓣特征提供了强区分信息。

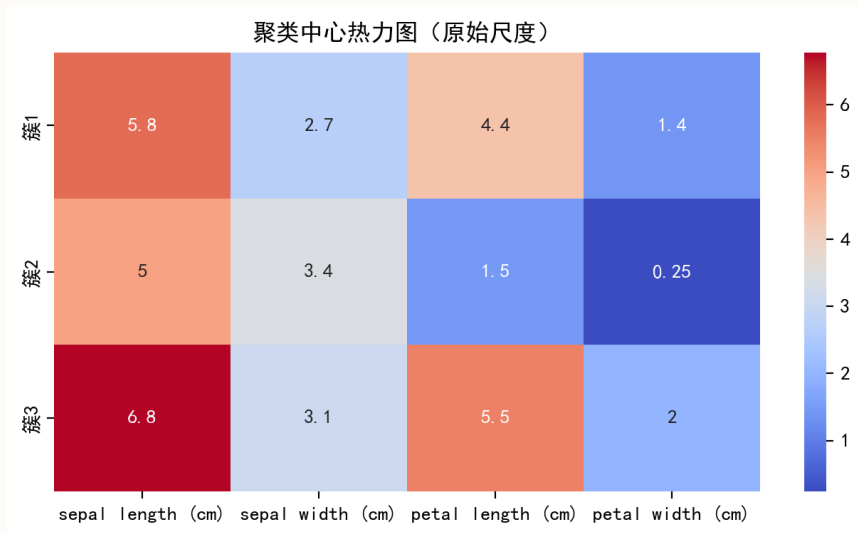
轮廓系数 > 0.55 ，聚类效果显著提升。

3、不同K值的影响（K=2 vs K=3）及肘部法则

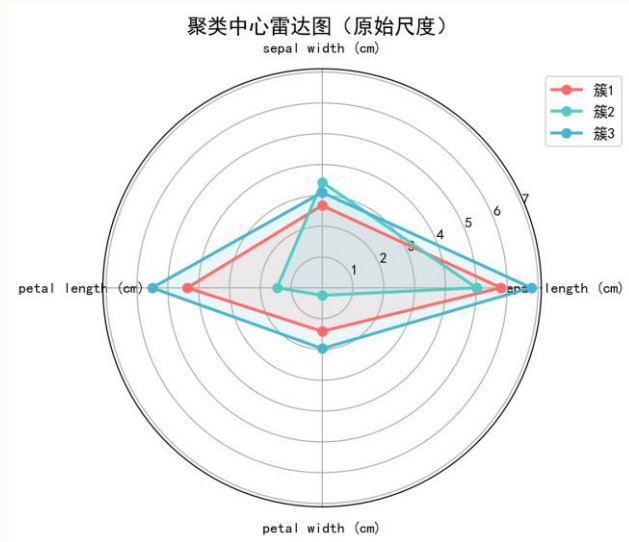


K=2时将簇1和簇3合并为一类，簇2单独一类，聚类比较粗糙。

4、聚类中心分析与可视化



K-means中每个簇的“质心”，是该簇所有样本在各特征上的平均值
对于4维数据，聚类中心是4维向量，可以反映每类鸢尾花的典型特征



簇2花瓣长度和宽度明显小于其他两类，花萼宽度较大；簇3各项特征均最大



数据挖掘与机器学习 作业汇总

信管2301

冯世欣

202321054001